Computers & Security 157 (2025) 104576

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

Behind the scenes of attack graphs: Vulnerable network generator for in-depth experimental evaluation of attack graph scalability

Alessandro Palma *, Silvia Bonomi

Department of Computer, Control, and Management Engineering "Antonio Ruberti", Sapienza University of Rome, Via Ariosto 25, Rome, 00185, Italy

ARTICLE INFO

Dataset link: https://github.com/ds-square/vuln-net-ag

Keywords:
Attack graph
Dataset generation
Experimental evaluation
Complexity analysis

ABSTRACT

An Attack Graph represents potential paths for attackers to compromise a computer network and security analysts use it to pinpoint vulnerable areas for cyber risk assessment. Due to their combinatorial complexity, designing scalable algorithms for generating these graphs without sacrificing their accuracy remains a challenge. Previous research focused on improving scalability, but evaluations often overlooked key parameters beyond network size, thus raising the natural question of their application in real-world settings. One of the main causes is the lack of data that the cybersecurity community faces in different areas, and cyber risk assessment in particular. To address this problem and support the comprehensive evaluation of attack graph algorithms, we introduce a dataset generator of vulnerable networks, which includes realistic reachability graphs and vulnerability inventories. This enables the design of an analytical framework to assess attack graph scalability comprehensively, considering diverse network and vulnerability dimensions. According to the proposed framework, we perform an in-depth experimental evaluation of the time and space complexities of attack graphs, offering novel insights into the critical parameters affecting them, and we extensively discuss how they inform and benefit future approaches.

1. Introduction

In modern networks, where the infrastructures are becoming more complex and cyber attacks more sophisticated, evaluating the exposure to cyber risk is crucial for any organization (Zhang et al., 2023). An Attack Graph (AG) is a graph-based threat model representing the possible paths that an attacker can follow to intrude and compromise a computer network or system. It is currently used by enterprises for threat analysis and risk management (Stergiopoulos et al., 2022): among the others, Security Information and Event Management systems (e.g., Skybox, Firemon) leverage AG for attack detection and monitoring from malicious traffic (Nadeem et al., 2022), while Cyber Threat Intelligence platforms use AGs to map out tactics, techniques, and procedures (Noel et al., 2023). Given its powerful analytics, the literature studied AG-based solutions for network hardening, security monitoring, and cyber risk assessment (Zenitani, 2022a). However, AGs come with the drawback of poor scalability due to their combinatorial size in the number of possible attack steps in a network (Kaynar, 2016). To mitigate this complexity, some of the aforementioned tools provide approximated AGs, e.g., considering only the shortest attack paths for prompt analysis but reducing the accuracy due to ignoring other potentially dangerous attacks. At the same time, researchers spent much effort to trade-off accuracy with computation time (Muñoz-González et al., 2019; Zenitani, 2022b).

Among the existing approaches, it is recognized that TVA (Jajodia et al., 2009), NetSPA (Ingols et al., 2006), and MulVAL (Ou et al., 2005) represent the three pillars from which many optimizations and improvements have been proposed (Nadeem et al., 2022; Palma and Angelini, 2024; Ibrahim et al., 2019; Li et al., 2020; Tayouri et al., 2023). The first problem related to these works and their subsequent developments is that they find their motivation in the theoretical computational complexity, and currently, few experimental studies have been performed to assess the scalability in different realistic network scenarios. The main reason for this poor investigation is the lack of datasets. To the best of the authors' knowledge, there are no available datasets to test AG algorithms, thus leading existing works to test approaches in very restricted scenarios and preferring theoretical proofs. As a consequence, the second problem is that most of these solutions focus on a single scalability dimension (i.e., the size of the network), overlooking other environmental information that may affect the AG scalability as well (e.g., vulnerabilities). This raises the natural question of their practical usability in real environments. This is related to a final emerging problem which is the lack of a systematic root cause analysis

* Corresponding author.
E-mail address: palma@diag.uniroma1.it (A. Palma).

https://doi.org/10.1016/j.cose.2025.104576
Received 18 February 2025; Received in revised form 16 May 2025; Accepted 14 June 2025
Available online 28 June 2025
0167-4048/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

of performance degradation, which hinders the possibility of designing scalable algorithms accordingly.

To address these problems, we first introduce a vulnerable network generator that creates a large dataset of network reachability graphs and corresponding vulnerability inventories, which we refer to as *Vulnerable Networks*. We demonstrate the similarity between the generated vulnerable networks and real-world scenarios by comparing them with three real networks. We foresee its employment in different cybersecurity areas, particularly as an AG scalability investigation benchmark. In fact, on top of it, we propose an analytical framework to investigate the scalability of AG algorithms according to different dimensions of networks and vulnerabilities. Finally, we present an in-depth experimental evaluation that highlights the root causes behind the degradation of the three mainstream algorithms referred by the community (namely TVA, NetSPA, and MulVAL) (Barik et al., 2016; Yang et al., 2023). The root cause analysis aims to answer the following Research Questions (RQs) about AG scalability:

RQ1: To what extent does the number of vulnerabilities impact the scalability of attack graph generation?
RQ2: Which are the network and vulnerability parameters affecting the attack graph scalability?
RQ3: Does the vulnerability distribution impact the attack graph scalability?
RQ4: How does the attack path computation scale?
RQ5: What is the interplay between vulnerability distribution and vulnerability diversity?
RQ6: How do the scalability results of the AG pillars apply to modern approaches?

We initially restrict the analysis to the three pillars. Then, in RQ6, we show how our findings apply to modern approaches by investigating recent works that still encapsulate these models' key aspects. For this purpose, we present a systematization of the AG experimental settings used in the literature. Finally, we discuss the informativeness and benefits provided by the contributions of this paper. In summary, we contribute:

(i) a vulnerable networks generator and its evaluation of the representativeness of real networks;
(ii) an analytical framework to validate AG generation algorithms, which includes a set of parameters that consider both network and vulnerability aspects;
(iii) novel comparative analyses of AG generation algorithms that provide new insights about the parameters affecting the scalability;
(iv) a systematization of the AG experimental settings in literature and their relations with the three pillars;
(v) open-source code for the dataset generator, AG generation, and the experimental evaluation.[1]

2. Background

In this section, we provide the fundamental notions of the Attack Graph (AG) model and the details of the three pillar algorithms used for the experimental evaluation. There exist two AG families: (i) *Topological AGs* model the attacker states according to the network topology (Jajodia et al., 2009; Ingols et al., 2006; Feng et al., 2017; Li et al., 2022a); (ii) *Logical AGs* model the dependencies of attacker states through logical axioms (Nadeem et al., 2022; Ou et al., 2005; Stan et al., 2022).

Over the years, different AG algorithms have been proposed (Kaynar, 2016; Palma and Angelini, 2024), and all of them take in input a *network reachability graph* and the system *vulnerability inventory*. A network reachability graph is a graph $G = (V, E)$ where nodes are associated with network hosts and edges represent reachability conditions between hosts (taking into account firewall rules and routing information). A vulnerability inventory is a list of network hosts with an associated set of vulnerabilities, typically obtained by vulnerability scanners (e.g., Nessus[2]).

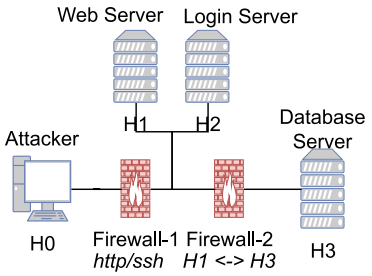


Fig. 1. Simple network configuration.

Table 1
Vulnerability inventory for the illustrative example.

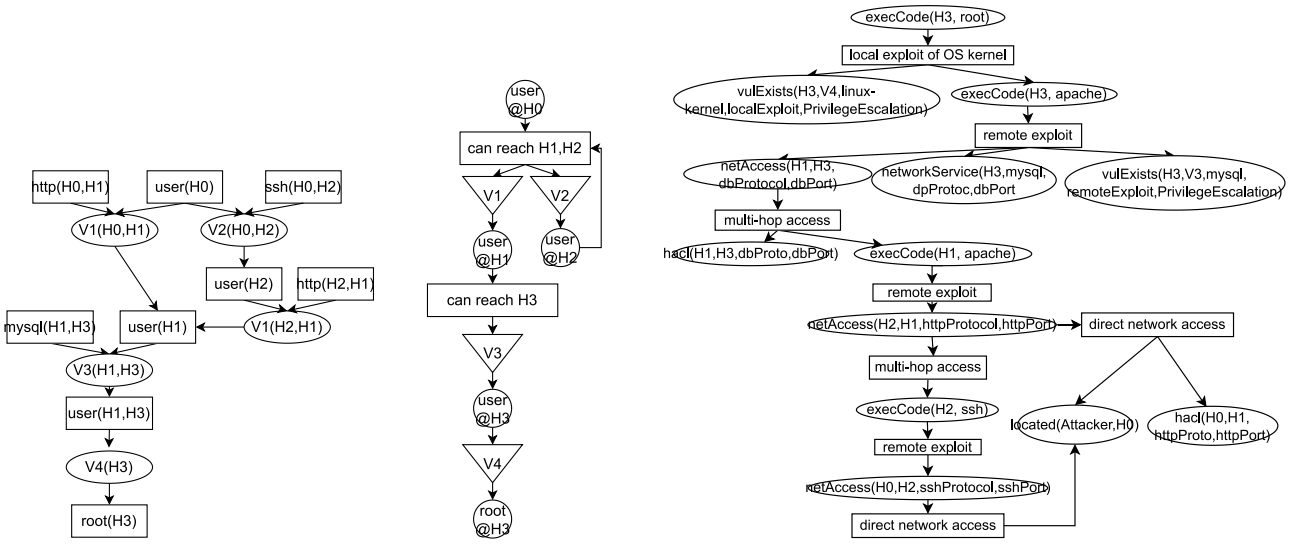
Software	Host	Vulnerability
Apache v2.1.0	H1	V1 (CVE-2023-37941)
SSH (Cisco IOS XR v7.4.1)	H2	V2 (CVE-2021-34718)
Oracle MySQL v8.2.0	H3	V3 (CVE-2024–20960)
Linux Kernel v5.15	H3	V4 (CVE-2019-3819)

Starting from these inputs, we can distinguish two main phases of the AG generation process: *Attack Graph Model Computation* and *Attack Path Computation*. The former defines the semantics of AG nodes and how to represent attack steps through AG edges, i.e., how to represent capabilities that an attacker needs to successfully perform an exploit, namely *pre-conditions*, and its consequent attacker state, namely *post-conditions*. Once the AG model is defined and instantiated, we need to compute (possibly all) the existing attack paths to perform security analysis and estimate the level of exposure to cyber risks or to monitor online the evolution of possible attacks. These paths represent sequences of compromised devices (or network segments) together with vulnerabilities exploited during the attack, and they are typically used to compute risk scores (Feng et al., 2017; Yuan et al., 2020; Sun et al., 2021). Different approaches exist in the literature for both AG modeling and attack path computation (Nadeem et al., 2022; Ibrahim et al., 2019; Li et al., 2022b; Palma and Bonomi, 2023) and all of them are based on three main models that are the pillars of the modern solutions: TVA (Jajodia et al., 2009), NetSPA (Ingols et al., 2006), and MulVAL (Ou et al., 2005).

In this section, we describe these approaches in detail as they are the basis for the proposed experimental evaluation. To compare them, we refer to the simple network configuration of Fig. 1 inspired by Barik et al. (2016) with the vulnerability inventory of Table 1. It consists of two firewalls protecting the Demilitarized Zone (DMZ), which includes a Web Server (H1) and a Login Server (H2). Firewall-1 protects the DMZ from the external network, where the attacker is located (in host H0). Firewall-2 protects the DMZ from the internal network, which includes a Database Server running on host H3. Firewall-1 allows HTTP and SSH traffic to Web and Login servers and blocks all other traffic. Firewall-2 allows access to the Database server only from the Web server. The vulnerability inventory related to the presented network is reported in Table 1: Host H1 is running a vulnerable version of the Apache web server, which has a vulnerability (CVE-2023-37941) that allows remote code execution if an attacker injects a malicious Python object through the two open ports of the host. The host H2 (Cisco IOS XR) exposes an open SSH port that enables the exploitation of a vulnerability (CVE-2021-34718), which allows an authenticated, remote attacker with

[1] https://github.com/ds-square/vuln-net-ag

[2] https://www.tenable.com/products/nessus



(a) TVA Attack Graph. (b) NetSPA Attack Graph. (c) MulVAL Attack Graph.

Fig. 2. Example of (a) TVA, (b) NetSPA, and (c) MulVAL models for the illustrative example.

low-level privileges to read arbitrary files and gain user privileges. Database server H3 is a Linux box running Oracle MySQL, which has a remotely exploitable vulnerability (CVE-2024-20960) through the open MySQL port, enabling a low-privileged attacker to cause a hang of the Server, while the Linux kernel in host H3 has a vulnerability (CVE-2019-3819) that allows local users to gain root privileges.

Topology Vulnerability Analysis (TVA). It is a topological AG that considers the dependencies between the possible exploits and the network assets (Jajodia et al., 2009). We refer to it as *Exploit Dependency Graph* and Fig. 2(a) shows an example of such a model, which has two types of nodes:

- *security condition nodes* represent the attacker's pre- and post-conditions for each exploit (rectangular nodes in Fig. 2(a)). For example, the node ⟨*http(H0,H1)*⟩ indicates the accessibility condition of web service in host H1 from host H0, while the node ⟨*user(H0)*⟩ indicates that the attacker has user privileges in host H0.

- *exploit nodes* represent the exploitation of vulnerabilities (oval nodes in Fig. 2(a)). For example, node ⟨*V1(H0,H1)*⟩ indicates that by exploiting vulnerability V1, the attacker moves from host H0 to H1.

Direct edges from security conditions to exploit nodes are the pre-conditions the attacker must meet to perform the exploit; direct edges from exploit to security conditions nodes are the post-conditions gained after the exploit. More formally:

Definition 1 (*Exploit Dependency Graph*)**.** An Exploit Dependency Graph $G = (V, E)$ is a directed graph in which $V = \{S \cup X\}$, where S is the set of security condition nodes and X is the set of exploit nodes, and $E = \{E_{sx} \cup E_{xs}\}$ such that E_{sx} are the edges where s is precondition for the exploit of vulnerability x, and E_{xs} are the edges where the exploit of vulnerability x has post-condition s.

The TVA generation algorithm is made up of two steps. In the first phase (*forward phase*) the algorithm considers the set of initial conditions S_{init} (i.e., initial attacker capabilities) and, based on topological links, finds the conditions whose pre-conditions match the post-conditions of S_{init}, iterating this procedure without revisiting the nodes twice. This first phase results in the graph G_{init} with all the forward dependencies from conditions S_{init}. In the second phase (*backward phase*), the algorithm considers the set of attacker goal conditions S_{goal} and performs a backward traversal of G_{init}. The resulting graph is the AG with all the necessary and sufficient sets of exploits for the initial and goal conditions. TVA algorithm has a theoretical computational cost of $O(N^2)$ with N being the number of network hosts. In the example of Fig. 2(a), the initial conditions are the rectangular nodes without in-edges, while the goal condition is the root privilege on host H3. To compute the possible attack paths given the resulting AG, TVA performs a backward traversal from the attacker goal conditions S_{goal} and substitutes each traversed exploit with the conjunction of its preconditions, resulting in a disjunction of conjunctions.

Network Security Planning Architecture (NetSPA). It is a topological AG that introduces the notion of prerequisite being either a reachability condition or a required access level (Ingols et al., 2006). We refer to it as *Multiple Prerequisite Graph* and Fig. 2(b) reports an example of the network configuration of Fig. 1. It has three types of nodes:

- *state nodes* represent an attacker's level of access on a particular host (oval nodes in Fig. 2(b)). For example, the node ⟨*user@H0*⟩ indicates that the attacker has user privilege at host H0.

-*prerequisite nodes* represent reachability groups (i.e., subnetworks) (rectangular nodes in Fig. 2(b)). For example, the node ⟨*can reach H1, H2*⟩ indicates that host H0 can reach H1 and H2.

- *vulnerability nodes* represent a particular vulnerability instance (triangular nodes in Fig. 2(b)). For example, the node ⟨*V1*⟩ indicates the exploit of vulnerability V1.

Direct edges from state to prerequisite nodes are the capabilities enabled by such states; the ones from prerequisite to vulnerability nodes are the pre-conditions for the exploit, and from vulnerability to state nodes are the states the attacker gains after the exploit.

Definition 2 (*Multi Prerequisite Graph*)**.** A Multi Prerequisite Graph $G = (V, E)$ is a directed graph in which $V = \{S \cup P \cup X\}$, where S, P, X are respectively the set of *state*, *prerequisite*, and *vulnerability* nodes, and $E = \{E_{sp} \cup E_{px} \cup E_{xs}\}$ such that E_{sp} are the edges from a state node s to the prerequisite p an attacker may exploit; E_{px} are the edges from the prerequisite p required for the exploitation of vulnerability x; E_{xs} are the edges $e = (x, s)$ from vulnerability node x to the state s reached after the exploitation.

NetSPA leverages the Breadth-First Search (BFS) algorithm (Lee, 1961). Starting from the set S of initial attacker states stored in a queue, it iteratively adds the nodes reachable from the currently visited node in the queue, upon the queue being empty, i.e., at most all the nodes are visited once. The resulting graph is the AG with the sets

of all exploits, considering initial and goal conditions enriched with information about the reachability groups. NetSPA algorithm has a theoretical computational cost of $O(max(V,T) \cdot R \cdot C$ with V, T, R, C being respectively the number of vulnerabilities, ports, reachability groups, and credentials. This could be expressed in the number of states S as $O(S \cdot lgS)$. In the example of Fig. 2(b), the initial condition is user privilege at host H0, while the goal condition is the root privilege at host H3. To compute the possible attack paths given the resulting AG, NetSPA performs the BFS traversal starting from the attacker's initial conditions.

Multihost, Multistage Vulnerability Analysis (MulVAL). It is a *Logical Attack Graph* that uses Datalog to express logical axioms describing network configuration, security policy, and vulnerabilities and *rules* which define the interactions between components in the network (Ou et al., 2005). Based on these elements, MulVAL defines:

- *Fact nodes* represent the asset state or network condition that must exist for the attacker to exploit the vulnerability (oval nodes in Fig. 2(c)). For example, the node ⟨*execCode(H3, root)*⟩ indicates the attacker can execute code on host H3 with root privileges.

- *Derivation nodes* represent the interaction rules the attacker must perform to attack (rectangular nodes in Fig. 2(c)). For example, the node ⟨*local exploit of OS kernel*⟩ represents the action that the attacker must perform to reach root privileges on host H3.

Direct edges from fact to derivation nodes and vice versa are the logical *"depends on"* relation.

Definition 3 (*Logical Attack Graph*)**.** A Logical Attack Graph $G = (V, E)$ is a directed graph in which $V = \{F \cup D\}$, where F, D are the set of *facts* and *derivation* nodes, and $E = \{E_{fd} \cup E_{df}\}$, where E_{fd} are the edges indicating that derivation d depends on fact f; E_{df} are the edges indicating that fact f depends on derivation d.

Before generating the attack paths, the MulVAL reasoning engine translates the attacker's goals into a set of *trace steps*, which are logical formulas expressing an attack step. A trace step is of the form: *Conjunct⇒Fact*, where *Conjunct* is a list of facts that must all be satisfied for the exploit, *Fact* is the specific attacker goal, and the whole formula expresses a MulVAL *rule* for an attack step. Then, the algorithm starts from the set of attack steps T and for each attack step, its derivation nodes N_r are figured out (i.e., the nodes composing the trace step). By definition, the facts N_f in the *Fact* clause are the parents of the derivation nodes N_r, while the facts N_c in the *Conjunct* clause are the children. By iterating the procedure over all the attack steps, the algorithm builds the logical AG by chaining formal clauses. The resulting graph is the AG with the sets of all logical clauses that must be satisfied to reach the attacker's goal. MulVAL algorithm has a theoretical computational cost of $O(N^3)$ where N is the number of network hosts. In the example of Fig. 2(c), the goal condition is the code execution with root privilege at host H3, while the initial conditions are the attack location in host H0 and the HTTP communication with host H1. To compute the possible attack paths given the resulting AG, MulVAL uses the Prolog engine XSB.

3. Related work

Different approaches proposed solutions to address the scalability problem. Table 2 provides a systematization of the current literature and the experimental settings used to validate AG algorithms. Specifically, the table highlights the following aspects: (i) the AG model on which the approach is based, (ii) the maximum number of hosts and vulnerabilities considered for scalability, (iii) the network topology tested, (iv) whether the experiments vary the vulnerability inventory, (v) whether the approach is validated on real data, (vi) whether the code is publicly available, and (vii) which solution is proposed to address the AG scalability problem. All the works analyze the AG scalability varying the number of hosts, considering it as the main parameter affecting the AG scalability. However, 70% of them ignore the number of vulnerabilities that are fixed and not used for scalability investigation. This underestimates the AG complexity because the AG model size increases with both the number of hosts and vulnerabilities (see Section 2). Among these works, three are notably different as they consider the number of vulnerabilities in the experiments (Hong and Kim, 2016; Ibrahim et al., 2019; Sabur et al., 2022). Thus, they recognize the impact of vulnerabilities on AG models, although they overlook other parameters like network topology.

In fact, 60% of the papers use a LAN-based topology, limiting the scenarios of AG scalability. Based on the different topologies, the number of attack paths may increase. For example, a mesh topology will have many more attack paths than a star topology, resulting in different scalability performance. Additionally, 43% of the works do not consider or specify the topology in the experiments, thus overlooking its effects on AG. Only *two* works consider different topologies for validation (Hong and Kim, 2016; Palma and Angelini, 2024), although they do not catch the fine-grained differences between the different topologies. The vulnerability inventories are specified in only 28% of the works, although they are fixed and not considered for performance evaluation. As different works propose heuristics based on vulnerability inventories (e.g., by aggregating vulnerabilities with the same risk values Bonomi et al., 2025), their consideration for AG scalability becomes important. Finally, *six* works tested their approach with real networks, while all the others use synthetic ones, and only *three* works have publicly available code, thus hindering reproducibility.

According to this analysis, there is a notable gap in the literature concerning AG scalability. While many existing studies focus on improving the scalability of AG generation with respect to the number of network hosts, significantly fewer studies address scalability in terms of vulnerabilities, network topology, and vulnerability inventories. This gap limits the real-world applicability of current AG models, especially for large and complex enterprise environments. All the existing works optimize AG algorithms primarily by reducing computational overhead tied to the number of network hosts. Techniques such as graph pruning, state-space reduction, and parallelization are employed to handle the combinatorial explosion resulting from large host counts. Only a limited number of works (30%) delve into how the number and diversity of vulnerabilities affect scalability. In real networks, hosts may have multiple overlapping vulnerabilities. Their modeling and analysis can be computationally expensive due to interdependencies, conditional exploitability, and a combinatorial explosion of attack paths in small systems with many vulnerabilities. Neglecting these factors in scalability studies leads to models that may underestimate the real computational burden of accurate AG generation. Additionally, network topologies are modeled simplistically in 83% of the papers, with limited consideration of different routing, firewalls, and subnet configurations. However, topological features may impact attack path computation, and their omission can skew both the performance and realism of generated AGs. In this paper, we investigate this research gap and quantify the impact of the network and vulnerability parameters through an extensive experimental evaluation.

In fact, to the best of the authors' knowledge, a comparative root cause analysis of AG scalability does not exist. One of the main reasons is the lack of datasets to validate AG algorithms. Different works addressed the problem of dataset generation in different cybersecurity areas, like code signature (Zhang et al., 2024), IoT rule set (Hashmat et al., 2022), automotive scenarios (Ding et al., 2021), and smart contracts (Rodler et al., 2023). Among them, the ones on network scenario generation are the most relevant for our purposes (Schreuders et al., 2017; Venkatesan et al., 2019). Schreuders et al. (2017) proposed a tool to build Virtual Machines (VMs) on randomized scenarios that are mainly used for educational purposes. Venkatesan et al. (2019) present a toolset to create one network scenario starting from network details (e.g., IP addresses, communication protocols). Given their focus on educational tasks, they are difficult to employ for benchmarking

Table 2
Experimental settings in the AG literature. *Max hosts* is the number of hosts; *max vuln per host* is the number of vulnerabilities; *topology* is the network topology; *vuln. inventory* marks if diverse inventory configurations are considered; *real data* marks if the approach is tested on real settings; *code* marks if the code is reproducible; *approach* indicates the solutions for scalability.

	AG model	Max hosts	Max vuln per host	Topology	Vuln inventory	Real data	Code	Approach
Kaynar and Sivrikaya (2016)	NetSPA	500	~10	lan-based	×	×	×	distributed
Wang et al. (2016)	TVA	600	5	×	×	✓	×	heuristics
Hong and Kim (2016)	NetSPA	1600	200	multiple	×	×	×	heuristics
Feng et al. (2017)	TVA	1000	5	mesh	×	×	×	heuristics
Li et al. (2017)	NetSPA	400	×	lan-based	×	×	×	distributed
Mjihil et al. (2017)	MulVAL	16	50	×	✓	✓	×	distributed
Ibrahim et al. (2019)	NetSPA	1000	270	×	✓	✓	✓	heuristics
Yousefi et al. (2018)	MulVAL	6	4	lan-based	×	×	×	alert-based
Yichao et al. (2019)	NetSPA	50	5	lan-based	×	×	×	alert-based
Yuan et al. (2020)	NetSPA	5	5	lan-based	✓	×	×	alert-based
Li et al. (2020)	TVA	15	5	×	×	×	×	distributed
Sun et al. (2021)	NetSPA	10	3	lan-based	×	×	×	heuristics
Nadeem et al. (2022)	MulVAL	2138	×	×	✓	✓	×	alert-based
Sabur et al. (2022)	MulVAL	4	250	×	×	×	×	distributed
Salayma (2024)	TVA	8	3	lan-based	✓	×	×	distributed
Palma and Angelini (2024)	TVA	250	250	multiple	✓	✓	✓	progressive
Bonomi et al. (2025)	TVA	250	50	lan-based	✓	✓	✓	heuristics

evaluations, and in particular for AG scalability, because they lack a systematization of network and vulnerability features during dataset generation. In contrast, we propose a vulnerable network generator capable of creating large datasets of networks with vulnerabilities, which can support the need for data in different tasks beyond AG scalability (e.g., attack detection).

Concerning this, a few works moved a step towards comprehensive AG evaluation. Lippmann et al. (2005) propose an experimental setting in which they tested NetSPA by scaling the number of hosts and four network topologies: flat, enclave, enterprise, and customized. They specify the number of vulnerabilities per host, open ports, and vulnerability types, although they are not used to test the scalability. Similarly, Heberlein et al. (2012) proposed a taxonomic foundation for comparing and contrasting AG-based approaches. They underline the inconsistencies among the different approaches, making evident the diverse assumptions, algorithms, and vulnerability conditions modeling. Regarding the performance, they examined the theoretical complexity claimed by the different works, without any experimental validation or reproduction of the results, as we proposed in this paper. Finally, Hong and Kim (2016) propose a multi-layer approach for AG generation and evaluate its theoretical complexity and scalability on different topologies and network sizes.

While these works are distinguished for considering not only network size, it is currently missing an in-depth investigation of the parameters affecting AG scalability and their correlation, as we propose in this paper, to inform the design of more scalable approaches.

4. Overview of the approach

To address the problem of the lack of data and the consequent poor AG scalability investigation, we present a general approach depicted in Fig. 3. The initial step is the design of a **Vulnerable Network Generator** to create a large dataset of reachability graphs and vulnerability inventories based on state-of-the-art information and resembling real networks. It benchmarks various network and vulnerability parameters, and the output is the first contribution of this paper and consists of a dataset that can be employed for different cybersecurity tasks, such as evaluation of vulnerability scanners, training for IDS, vulnerability prediction, and other similar approaches that suffer the lack of this data (Sadlek et al., 2022; Palma and Angelini, 2024; Presekal et al., 2023).

In particular, we employ the resulting dataset for the design of an **AG Analytical Framework** that provides a standard and systematic approach for the evaluation of AG algorithms. Being based on a comprehensive dataset, the analytical framework is capable of furnishing an extensive investigation of AG scalability based on empirical time and space complexity. Let us note that while the current literature focuses on their theoretical analysis, the analytical framework contributes to a practical investigation.

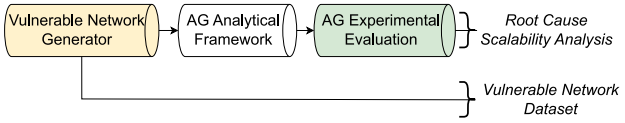


Fig. 3. Proposed approach for AG scalability evaluation.

To this aim, we leverage the framework for the **AG Experimental Evaluation** to analyze the root causes of scalability degradation of the three pillar algorithms. It results in an extensive and systematic collection of findings that we present in terms of research questions related to AG scalability. They enable more informativeness for the design of future algorithms that improve the scalability of AGs, which we discuss in Section 7.

4.1. Vulnerable network generator

The vulnerable network generator must provide a comprehensive dataset of reachability graphs and vulnerability inventories for a deep analysis of AG algorithms. The former can vary in size and topology; the latter can vary in size and diversity (i.e., the variety of vulnerabilities and their required privileges). Let us note that we are interested in the dependency between a reachability graph and its corresponding vulnerability inventory, which must be considered together. To this aim, we refer to such interdependency as a *Vulnerable Network*.

Given that vulnerabilities are associated with different hosts, combining the two inputs introduces additional control parameters such as vulnerability distribution (i.e., how the vulnerabilities are distributed among the network hosts). In light of these considerations, we notice that AG complexity is dependent on the following parameters:

(i) the number of hosts (Zenitani, 2022a), as it determines the size of the input reachability graph;
(ii) the reachability graph topology (Griner et al., 2021), given that the AG is computed starting from the reachability graph, it impacts the number of potential attack edges;
(iii) the number of vulnerabilities (Sabur et al., 2022), as it determines the size of the vulnerability inventory input;

(iv) the diversity in the vulnerability inventory (Ingols et al., 2006), as the number of vulnerability nodes may vary according to the number of *unique* vulnerabilities;
(v) the distribution of vulnerabilities among hosts (Amankwah et al., 2020), as it reflects the distributions of AG nodes and edges.

To give the rationale for these parameters, let us recall that every considered AG model has at least one node per host and one node per vulnerability; thus, their size is crucial for scalability. Concerning the network topology, it is possible to observe that it represents the *skeleton* of the AG and thus it represents a lower bound of the potential number of nodes and edges, indicating the size of the AG and its paths (intended as combinations of attack steps). In addition, vulnerabilities play a crucial role in AG models not only from the numerical point of view but also from the distribution and diversity ones, as they may introduce multiple edges depending on the specific combinations of pre- and post-conditions related to the exploit.

For these reasons, we design the vulnerable network generator according to the following specifications.

- The number of hosts varies in a range between 10 and 1000, to analyze the scalability from small to large networks;
- The number of vulnerabilities per network host varies between 10 and 500 to analyze the scalability from less to more vulnerable systems according to state-of-the-art analytics (Upadhyay and Sampalli, 2020; Open Web Application Security Project, 2021);
- The following network topologies are considered:
  - *Mesh*: it is a complete graph and simulates a network where all the hosts can reach any other host (e.g., internal LAN) (Sichitiu, 2005).
  - *Star*: it simulates a central node in the network (e.g., a router of a LAN) acting as an intermediary for communication (Sheng et al., 2021).
  - *Ring*: it simulates circular communication of a LAN/WAN (Jain et al., 2022).
  - *Erdos–Renyi random graph* (Erdős and Rényi, 1959): it is constructed by connecting nodes randomly with probability $p = 0.5$, independently from every other edge.
  - *Power-law* graph: it represents the common topology of today's networks according to studies that have shown that Internet graphs and network systems obey the power law (Palmer and Steffan, 2000; Knight et al., 2011), which considers out-degree, number of hops, and eigenvalues.
  - *LAN-based*: it represents a custom network composed of four subnetworks: a Demilitarized Zone (DMZ), an Administrative LAN, and two internal LANs. We consider this network setting as it is used in many existing AG works (Sun et al., 2021; Kaynar and Sivrikaya, 2016; Ou et al., 2005), and we investigate the reachability between the subnetworks by varying the percentage of links of 0%, 25%, and 50% of all the possible links.
- The different vulnerability distributions among network hosts vary according to the most known discrete distribution. This parameter is important as typically not all the hosts have the same and uniform number of exploitable vulnerabilities:
  - *Uniform distribution*: it is the case where all the hosts have the same number of vulnerabilities.
  - *Pareto distribution*: it is the case in which half of the hosts have 80% of the vulnerabilities and the other half the 20%.
  - *Binomial distribution* with probability $p = 0.5$: it is the most common distribution in real-world scenarios (e.g., in network traffic).
  - *Poisson distribution*: it simulates the arrivals of new vulnerabilities according to the parameter $\lambda = 1$, indicating one new vulnerability at a time.
- The vulnerability diversity among the network hosts varies from cases where all the hosts have the same vulnerabilities (e.g., replicas in distributed systems) to cases where the hosts have all different vulnerabilities (e.g., different servers of DMZ). We vary vulnerability diversity from 0% (all the hosts have the same vulnerabilities) to 100% (none of the hosts have the same vulnerabilities) with steps of 25%.

The above parameters are combined according to a benchmarking approach (Jain, 1991) that considers all their possible combinations. Given that we consider the number of hosts with steps of 100 hosts, and the number of vulnerabilities with steps of 25 vulnerabilities, the resulting dataset includes $n = 10$ configurations for the number of hosts, $v = 20$ for the number of vulnerabilities, $t = 6$ topologies, $d = 4$ distributions, and $r = 5$ configurations of diversity. Thus, the benchmark is made of $n \cdot v \cdot t \cdot d \cdot r = 10 \cdot 20 \cdot 6 \cdot 4 \cdot 5 = 24{,}000$ vulnerable networks (i.e., reachability graphs and vulnerability inventories).

We are particularly interested in capturing realistic components in synthetic network scenarios. While the benchmark specifications include an exhaustive combination of parameters (i.e., all well-known topologies and discrete probability distributions), a point worth discussing is the type of vulnerabilities present in the different hosts. For example, if a host represents a Windows laptop, then it should contain vulnerabilities accordingly and not be randomly chosen. To this aim, we associate with each node in the reachability graph a collection of platforms from the standard Common Platform Enumeration (CPE)[3] knowledge base. Each platform in the knowledge base has an associated set of vulnerabilities (CVEs) that form the synthetic vulnerability inventories, adjusted to meet the size, distribution, and diversity requirements of the benchmark. Concerning the type of platforms and corresponding vulnerabilities, we selected a subset of services that are considered by the current literature (Jajodia et al., 2009; Ingols et al., 2006; Yuan et al., 2020; Sun et al., 2021; Sabur et al., 2022; Kaynar and Sivrikaya, 2016; Wang et al., 2016), although they can be easily customized. They include the following components:

- Operating systems: Windows 10, Ubuntu Linux 22.10, and Debian Linux 11. They include the corresponding components, such as office tools and file management platforms;
- Database management systems: MySQL, PostgreSQL, and Neo4j;
- Cloud servers: Azure and AWS;
- Web servers: Apache, HTTP Web Server, and nginx.

Let us note that including all possible services existing in the world is impractical, given their number and diversity. The goal of the generator is to be representative of the most common real network settings. To this aim, we investigate and validate the representativeness of real network settings considering *three* real scenarios.

4.1.1. Representativeness of real networks

Since the synthetic vulnerable networks drive the quantitative evaluation of the performance of AG algorithms, it is important to show their capability of representing real scenarios. We start from *three* real networks and perform similarity analysis between the parameters of the proposed generator and the ones from the real networks.[4]

The similarity analysis consists of the following steps:

1. We check whether the numbers of hosts and vulnerabilities are within the corresponding ranges set in the generator: it results in a boolean outcome (1 if within the range, 0 otherwise).

[3] https://cpe.mitre.org/

[4] Data about these real networks are the only ones available to the authors at the time of the writing of this manuscript. To the best of the author's knowledge, there are no other similar data for real networks available.

Table 3
Network and vulnerability parameters in the real networks and similarity analysis.

Parameter	IT-net		HC-net		IoT-net	
	Value	Similarity	Value	Similarity	Value	Similarity
Num. hosts	**200**	1	**21**	1	**40**	1
Avg. vuln. per host	**74**	1	**162**	1	**17**	1
Total vuln.	**14,875**	1	**3233**	1	**683**	1
Network topology	**LAN**	0.93	**Random**	0.66	**Star**	0.85
Vuln. distribution	**Binomial**	0.90	**Uniform**	0.71	**Poisson**	0.92
Avg. vuln. diversity	**0.87**	1	**0.75**	1	**0.50**	1

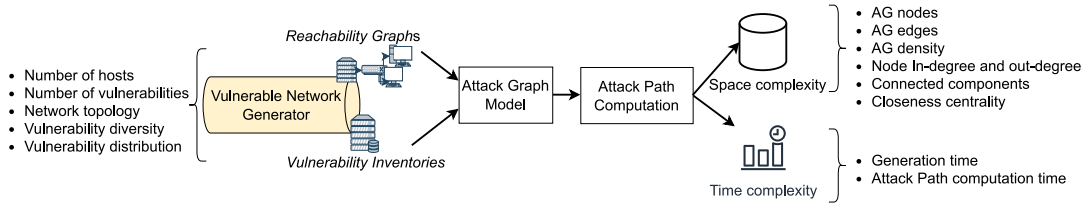


Fig. 4. Experimental framework for AG scalability evaluation.

2. We leverage graph similarity approaches (Zager and Verghese, 2008) to compare the topology of the real network with the ones proposed in the generator. We leverage the Graph Edit Distance (GED), normalized in the range $[0, 1]$ with min–max normalization, which measures the sequence of node/edge edit operations to make two graphs isomorphic.
3. We use the Chi-squared test (Mannan and Meslow, 1984) to compare the vulnerability distributions of the real network with the ones considered in the vulnerable network generator. The outcome is a value between 0 (totally different distributions) and 1 (equal distributions), representing the significance of the similarity.
4. We compute the Jaccard similarity (Tanimoto, 1958) between any pair of hosts to evaluate the vulnerability diversity percentage. Finally, we check whether it is within the ranges set in the generator, resulting in a boolean outcome (1 if within the range, 0 otherwise).

In the following, we present the real (anonymized) networks and similarity analysis. Results are in Table 3.

IT-net. It is the IT network of a university department with 200 devices arranged in $five$ LANs, corresponding to the four department areas plus the DMZ. The vulnerability inventory has been collected by running the Nessus and OpenVAS vulnerability scanners and manually removing the false positives from them. The resulting vulnerability inventory consists of 14,875 vulnerabilities in the whole network, with an average of 74 vulnerabilities per host. This IT network has been explored and (anonymously) released in previous works related to AG scalability (Palma and Angelini, 2024; Bonomi et al., 2025). The similarity analysis highlights that the most similar vulnerability distribution is the *Binomial distribution* with a similarity of 0.90 and the vulnerability diversity is 0.87 on average. According to GED, the most similar topology is the *LAN topology* with a similarity of 93%.

HC-net. It is a healthcare network from an italian hospital laboratory with 6 medical devices and 15 between servers and laptops. It is the result of the collaboration between researchers and healthcare operators during the international Panacea project.[5] The vulnerability inventory has been collected with the Nessus vulnerability scanner and manually removing false positives. The resulting vulnerability inventory consists of 3233 vulnerabilities in the whole network, with an average of 162 vulnerabilities per host. The similarity analysis highlights that the most similar vulnerability distribution is the *uniform distribution* with a similarity of 0.71, and the vulnerability diversity has a mean of 0.75. According to GED, the most similar topology is the *random topology* with similarity 66%.

IoT-net. It is a network emulating an automotive IoT scenario from an energy provider company. The network comprises 5 electric vehicles, 15 charging stations, and 20 devices of type data storage, web servers, and laptops. It represents the infrastructure of a Greek energy provider company as illustrated in Palma et al. (2025). We use Nessus and OpenVAS vulnerability scanners to discover vulnerabilities for each device and collect the vulnerability inventory. They reported a total of 683 vulnerabilities, with an average of 17 vulnerabilities per host. The similarity analysis highlights that the most similar vulnerability distribution is the *Poisson distribution* with a similarity of 0.92 and the vulnerability diversity is 0.5 on average. According to GED, the most similar topology is the *star topology* with 0.85 similarity.

Considering the parameter values of Table 3, we observe that in the real networks, there are many more vulnerabilities than the ones considered by existing works (see Table 2). They are between 17 and 162, while the most common setting in the literature is 10, up to 16 times less. Surprisingly, the vulnerability distribution is different: while most state-of-the-art solutions consider uniform distribution, the real networks include random and Pareto distributions as well. Similarly, the vulnerability diversity is always over 0.5, in contrast to the common setting in the literature that considers diversity 0 or not specified. Finally, the network topology varies among the different networks, as we expect, since it is strongly related to the services the network is built for. These discrepancies with the current literature highlight the need to have a comprehensive dataset to test AG algorithms, as current solutions may only partially cover real scenarios.

Additionally, we observe that the sizes of the network and vulnerability inventory are always captured by the synthetic network, and so is the level of vulnerability diversity. In fact, the ranges included in the generator are large enough to cover a very wide range of networks. On the other hand, the network topology included in the generator has a similarity between 0.66 and 0.93 with the real networks, with an average value of 0.81. Similarly, the vulnerability distribution has a similarity between 0.71 and 0.92, with an average value of 0.84. We consider these results satisfying for different reasons.

At first, real networks are rarely built according to a systematic topology but are adjusted to the stakeholders' needs. This means that a real topology corresponds to hybrid structures of well-known topologies. As proof, let us observe that the university department network (typically easier to manage) has the highest similarity (0.95) with LAN topology. In contrast, the healthcare network has the lowest one

[5] https://panacearesearch.eu/

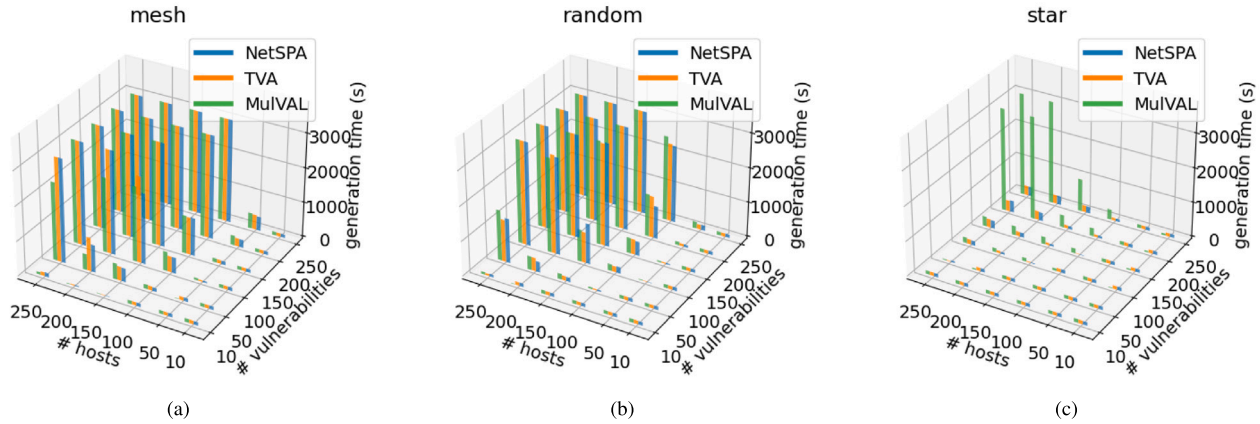


Fig. 5. AG generation time in (a) mesh, (b) random, and (c) star topologies. The x-axis is the number of hosts in the network, the y-axis is the number of vulnerabilities per host, and the z-axis is the generation time (in seconds). The highest bars are impractical computations.

(0.66) because it is the result of modifications necessary to comply with constraints that are commonly present in critical infrastructures.

A second observation is that in real scenarios, the vulnerability distribution is typically an uncontrollable parameter, in the sense that it cannot be handled systematically by network and security administrators because, for example, it cannot predict how many and where new vulnerabilities will appear. For the same reason, vulnerabilities in the network may follow no well-known distribution. In light of this consideration, the vulnerability distribution accuracy of the proposed generator (0.84) can be considered satisfying.

Let us note that we did not perform the similarity analysis of the type of platforms because every network has different services depending on its operations and, as previously discussed, representing all of them is impractical. As a final remark, to the best of the author's knowledge, there is no similar generator in the literature. This means the considered similarity analyses constitute a baseline for future and possibly more sophisticated vulnerable network generators. In light of these considerations, we believe that the obtained results are satisfying in terms of the representativeness of real network scenarios.

4.2. Analytical framework

Considering the vulnerable networks generated with the proposed approach, we introduce the analytical framework to investigate the scalability of AG algorithms, reported in Fig. 4. It takes as input the reachability graphs and vulnerability inventories and builds the AG models corresponding to the three pillars (TVA, NetSPA, and MulVAL). Consequently, it generates the attack paths running the different algorithms and finally empirically analyzes the space and time according to state-of-the-art metrics. Let us note that while these metrics are well-known in the literature, their relation with AG has not been extensively investigated yet. One of the contributions of the analytical framework is to give a rationale for evaluating AG algorithms, beyond the methodological approach. For what concerns the empirical space measurement, we consider the following metrics:

- Number of AG nodes and edges: they inform about the size of the AGs. We refer to $|V|$ and $|E|$ for expressing the number of nodes and edges, respectively.
- Density: the density of a graph (Coleman and Moré, 1983) informs about how much the number of its edges is close to the maximum, and it is $D = \frac{|E|}{|V|(|V|-1)}$ in directed graphs. AGs with higher density will have potentially more combinations of edges (i.e., attack steps) that result in more exploitable paths and expensive computations than sparse graphs.
- Node degree distribution: the degree of a node in a directed graph is the number of its in- and out-edges. Formally, let v be a node of a directed graph $G = (V, E)$, then we refer to $in_degree(v) = |E_{in}|$ to the number of in-edges of the node v and $out_degree(v) = |E_{out}|$ to the number of its out-edges. The distribution of the degree among the nodes of an AG informs about the rough number of paths passing through a node. Thus, the higher the average degree, the more the number of paths will grow because it derives from the combination of in-edges and out-edges of a node. In particular, this metric identifies the presence of isolated nodes (degree 0), i.e., unreachable attack steps, and dominating nodes (degree $|V|-1$), i.e., nodes where all the attack paths pass through.
- Number of connected components: the connectivity of a graph informs about the minimum number of nodes or edges that need to be removed to separate the remaining nodes into two (or more) isolated subgraphs. More precisely, connected components are the maximal connected subgraphs of a graph. The number of connected components in an AG may define a bound for the number of attack paths since all of them are within each component. This means that the higher the number of components, the fewer the combinations of attack paths between components.
- Node centrality distribution: closeness centrality of a node in a graph is the average length of the shortest path between that node and all other nodes in the graph and is calculated as $C(v) = \frac{|V|-1}{\sum_u d(u,v)}$, where $d(u, v)$ is the distance between u and v i.e., the minimum number of edges connecting them (Sabidussi, 1966). Thus, the more central a node is, the closer it is to all other nodes and the more the node is involved in different attack paths. To say differently, higher centrality will result in fewer combinations of different attack steps, as most of the paths pass through the central nodes.

Concerning the empirical computation time, we analyze the generation time taken by the AG model and the time needed to compute the attack paths, varying the number of attack sources and targets from 1 to 50. These space and time metrics cover the possible analyses that can be made using AG (Zenitani, 2022a) and are reported in Fig. 4.

5. Experimental evaluation of AG scalability

This section presents the experimental evaluation through the proposed framework and dataset generator, identifying the root causes of AG scalability. For each of the 24,000 settings (see Section 4.1), we implemented TVA and NetSPA according to their descriptions in the original papers and MulVAL with its open-source version. We analyze

Table 4
Space analysis of attack graphs according to different network topologies (mesh, random, and star).

	Mesh			Random			Star		
	NetSPA	TVA	MulVAL	NetSPA	TVA	MulVAL	NetSPA	TVA	MulVAL
num. nodes	7114	5086	7093	6953	4947	8554	1559	1554	797
num. edges	15,199	699,693	19,068	14,370	293,411	22,814	3057	3051	2239
density	0.002	0.04	0.001	0.002	0.01	0.001	0.005	0.005	0.01
num. strong components	228	3723	1965	457	4485	8168	1559	1554	772
median centrality	0.18	0.32	0.14	0.17	0.02	0.08	0.001	0.001	0.03

the space and time metrics of the framework (Fig. 4) and store the AGs in *graphml* format files, an extension of XML for graphs.

Let us premise that given the computational complexity of AG generation algorithms, we define thresholds for the computation time. Such thresholds may vary according to different scenarios. For example, if AG algorithms are used for the detection of multi-step attacks, then AG generation should be computed within seconds to minutes (Navarro et al., 2018). Similarly, for near real-time incident response and what-if analysis, the AG generation should be performed in minutes to one hour (Angelini et al., 2019; Gonzalez-Granadillo et al., 2018). Differently, for periodic and offline risk assessments, the AG generation is not urgent and can be computed in a few hours. In light of these considerations, we consider the most general scenario of incident response and what-if analysis (Zenitani, 2022a; Kaynar, 2016; Yang et al., 2023), thus we assume that when the AG generation does not terminate within one hour (3600 s), the problem is impractical. Given the higher complexity, for attack path computation, we assume that when the algorithm terminates within one hour (3600 s), the scalability is good; when it is between one and two hours (7200 s), the scalability is reasonable, otherwise, the problem is impractical. We conducted the experimental evaluation on a Linux server with an Intel(R) Xeon(R) Gold 6248 CPU 2.50 GHz and 256 GB of memory. In the following, we present the most relevant analyses as answers to different research questions (RQs).

RQ1: ***To what extent does the number of vulnerabilities impact the scalability of attack graph generation?***

Most of the literature evaluates AG scalability by varying the network size, but the number of exploitable vulnerabilities is also crucial for AG. Fig. 5 reports the generation time (in seconds) of the three different models increasing both the number of hosts (x-axis) and the number of vulnerabilities per host (y-axis) and considering different network topologies. For the sake of space, we report mesh, Erdos–Renyi random, and star topologies to represent the worst, average, and best scenarios, respectively. In this experiment, vulnerabilities follow the uniform distribution among hosts, and the inventory includes all different vulnerabilities (i.e., vulnerability diversity is 1). To complement the time analysis of Fig. 5, Table 4 indicates the average values of space metrics.

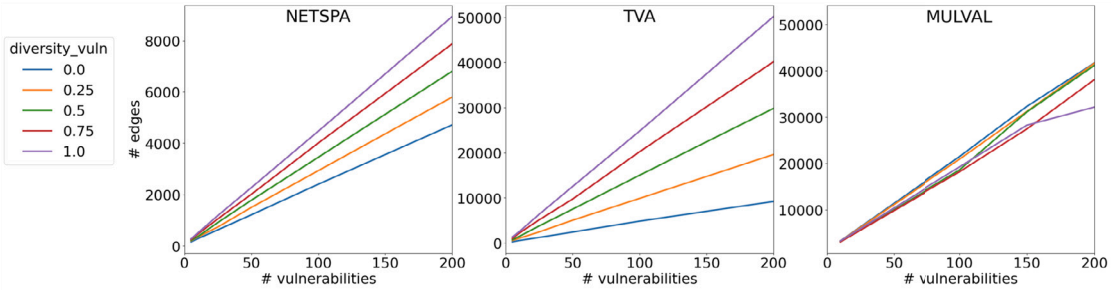
From the analysis of each plot in Fig. 5, we can notice that the variations in the number of hosts and vulnerabilities are comparable to each other. For example, in the mesh and random topologies (Figs. 5(a), 5(b) respectively), it is evident that the generation time increases rapidly when the number of vulnerabilities is greater than 50 and when the number of hosts is greater than 100. This indicates that the number of vulnerabilities impacts the AG scalability as much as the number of hosts, thus none of the two affects the scalability much more than the other. To this purpose, let us note that with networks and inventory sizes greater than 100 hosts and 100 vulnerabilities per host (with mesh and random topologies), the time to generate the AG model is more than 3000 s (i.e., about one hour). Looking at the trend of the bars along the x-axis, the size of the bars increases exponentially, and the same happens if considering the y-axis. This indicates that the generation time is exponential in both the number of hosts and vulnerabilities. Indeed, looking at both axes together (i.e., the x-y diagonal in the plots), the size of the bars increases twofold exponentially. Let us note that apparently, the generation time stabilizes in the settings with 150 hosts and 150 vulnerabilities, but it is only the effect of the predefined thresholds of intractability.

Another interesting finding arises from the comparison of the three plots in Fig. 5. It is notably evident that the time analysis is different if considering different topologies. The star topology (Fig. 5(c)) scales much better than the mesh and random ones, especially for the topological models (i.e., NetSPA and TVA). This is confirmed by the space analysis in Table 4: in the three models, the star topology leads to a number of nodes and edges being from 10% to 20% less than the mesh. In contrast, the random topology has a space complexity of just less than 4% on average than the mesh.

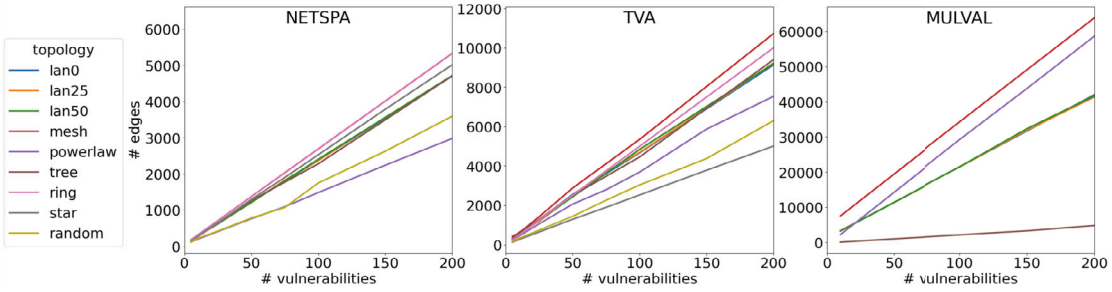
Result 1: *The number of hosts and vulnerabilities comparably affect the AG scalability: the AG generation of a small network with many vulnerabilities scales similarly to a big network with few vulnerabilities, according to an exponential trend. To enforce that the network size is not the main cause of poor AG scalability, let us note that the AG generation of networks with 250 hosts scales quickly if the number of vulnerabilities is just 10 per host.*

RQ2: ***Which are the network and vulnerability parameters affecting the attack graph scalability?***

Because neither the hosts nor the vulnerabilities can be ignored while studying the AGs, we investigate deeply how their features impact the scalability. To answer RQ2, we vary the diversity of the vulnerability inventories and the network topologies of the reachability graphs and study the space complexity through the number of edges in the resulting AGs (y-axis) by scaling the number of vulnerabilities (x-axis) as reported in Fig. 6. We consider 25 hosts and a uniform distribution of the vulnerabilities between hosts. To study the vulnerability diversity (Fig. 6(a)), we consider the *random* topology; to study the topology (Fig. 6(b)), we consider the vulnerability diversity 0. From Fig. 6(a) is discernible the high impact of the vulnerability diversity for the space analysis (and consequently the time too, given that attack path computation is proportional to the number of edges Kaynar, 2016). This is particularly conspicuous in topological models (i.e., NetSPA and TVA), for which the number of edges in the case of all different vulnerabilities (diversity 1) significantly surpasses the one with all equal vulnerabilities (diversity 0). Specifically, in NetSPA, the former scenario doubles the edges compared to the latter, while TVA is even worse, as this augmentation magnifies to sixfold proportions. This indicates that topological AG scalability is intricately proportional to the vulnerability diversity. The reason for this result is that by increasing the vulnerability diversity, the diversity of attack steps increases, and so does the number of edges. This augmentation, in turn, leads to more vulnerability exploits (and their combinations) and consequently more attack paths. Given that the number of possible paths is combinatorial with the number of attack steps (Kaynar, 2016), the vulnerability diversity is a pivotal parameter that highly impacts the AG scalability of topological models. Conversely, the logical AG (i.e., MulVAL) is almost invariant to the vulnerability diversity. This is because logical formulas are built such that an edge exists for each vulnerable service independently of its type.



(a) Vulnerability diversity analysis.



(b) Network topology analysis.

Fig. 6. Space analysis (number of edges) in relation to (a) vulnerability diversity and (b) network topology.

Result 2-A: *Vulnerability diversity matters: the number of hosts and vulnerabilities alone are inadequate for thoroughly examining AG scalability. For the topological AGs, the vulnerability diversity (i.e., the ratio between the number of unique vulnerabilities to the total) significantly impacts the scalability, increasing the edge count from twice to six times for higher diversity degrees.*

Looking at Fig. 6(b), we notice that MulVAL is more topology-sensitive than topological models, as we can expect given their emphasis on the topology. Topology-induced changes in MulVAL result in an increased number of edges, reaching a sixfold escalation. In particular, the mesh topology generates more than six times the number of edges as the tree topology. It is a result that we can expect because MulVAL is modeled as a tree, thus reflecting the network topology. In contrast, the topological models have a maximum difference of 1.6 times, and it is between mesh (∼2500 edges in NetSPA, ∼4000 edges in TVA) and power-law in NetSPA (∼1500 edges) and star in TVA (∼2500 edges).

Result 2-B: *Network topology matters: for the topological AGs, it impacts the scalability with the number of edges that may increase twice by having one topology (e.g., mesh) rather than another one (e.g., power-law/star). It is even more impacting for the logical AG, where the edges may increase six times from tree to mesh.*

RQ3: ***Does the vulnerability distribution impact the attack graph scalability?***

To proceed with the scalability analysis, we examine various vulnerability distributions among hosts, varying both the vulnerability distribution and their placement on the reachability graph. We consider a network setting with 100 hosts and 100 vulnerabilities per host with diversity 1 and three different distributions: *uniform* distribution, i.e., every host has the same number of vulnerabilities, *centrality-based Pareto* distribution, i.e., the 20% most central hosts have 80% of vulnerabilities, and *peripheral-based Pareto* distribution, i.e., the 20% most peripheral hosts have 80% of vulnerabilities.

Table 5 reports the most relevant results of time and space analyses. Concerning the AG model computation time, the centrality-based Pareto distribution requires approximately double the time of the other distributions, especially for topological models. This is because the most central hosts represent a bottleneck in generating all the dependencies of the related attack steps. In contrast, the path computation time scales much better when the most *central* hosts have more vulnerabilities (bold cell in Table 5). In these cases, most attack paths pass through such nodes, reducing the number of combinations of different attack steps with the other peripheral nodes. The data confirms this result as the centrality-based setting computes paths in 0.5 and 1.5 h for NetSPA and TVA, while the other cases are impractical.

Concerning the space analysis, the number of strongly connected components is of particular interest. From the uniform to the centrality-based configuration, it increases from 110 to 1488 in NetSPA and from 7960 to 8580 in TVA. Let us remark that the number of strongly connected components in an AG follows the rationale that the higher the number of components, the less the combination of attack paths between components (see Section 4.2). This indicates that centrality-based configurations lead to more disjoint sets of attack paths, corresponding to the different components. As a consequence, the combinations of attack paths for such a configuration are fewer than other distributions, resulting in more scalable computation, as confirmed by the path computation time. The reason for this behavior is that most of the attack paths (≈80%) pass through the 20% of central nodes and, once computed, fewer combinations remain with the peripheral connected components.

Finally, Table 5 shows that in the topological AG, the centrality of the networks reflects the centrality of the related AGs: in the uniform setting, the AG centrality is 0.5, in the centrality-based setting it is 0.8, and in the peripheral-based is 0.6. In contrast, although the logical AG has the worst performance, it is more invariant to the different distributions. This is because MulVAL generates the AG with logical formulas, losing the topological structure of the input reachability graph (as shown in Fig. 6).

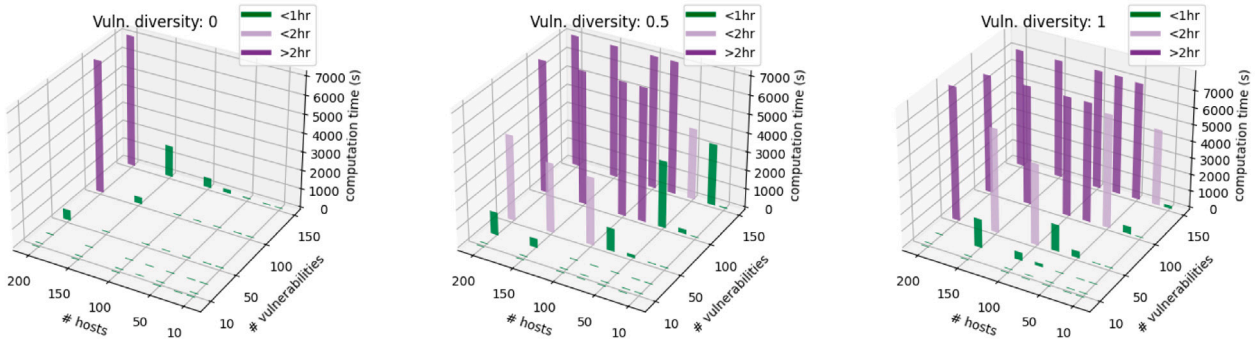
Result 3: *Distribution and placement of vulnerabilities matter: when the vulnerabilities are mostly in central parts of the network, the path computation has good or reasonable performance (≤1.5 h) than a peripheral or uniform distribution, which is impractical.*

RQ4: ***How does the attack path computation scale?***

Research questions RQ1, RQ2, and RQ3 address the analyses of the individual parameters affecting AG scalability, showing that the number of hosts, vulnerabilities, topology, diversity, and distribution all affect the scalability. However, a deeper investigation considers the computation of attack paths as the main analysis for cyber risk assessment and network hardening for which AGs are employed. We

Table 5
Analysis of vulnerability distributions.

		Uniform distribution	Centrality-based Pareto distribution	Peripheral-based Pareto distribution
TIME	AG Model computation time	Netspa: 186.53 s TVA: 190.21 s MulVAL: 204.38 s	Netspa: 408.34 s TVA: 413.65 s MulVAL: 419.76 s	Netspa: 190.62 s TVA: 272.29 s MulVAL: 341.28 s
	Path computation time	Netspa: >2 h TVA: >2 h MulVAL:>2 h	**Netspa: ~0.5 h** **TVA: ~1.5 h** **MulVAL: >2 h**	Netspa: >2 h TVA: >2 h MulVAL: >2 h
SPACE	Number of nodes	NetSPA: 9996 TVA: 9991 MulVAL: 38206	NetSPA: 10289 TVA: 10284 MulVAL: 39013	NetSPA: 10265 TVA: 10260 MulVAL: 38907
	Number of edges	NetSPA: 20141 TVA: 522400 MulVAL: 90825	NetSPA: 20756 TVA: 960951 MulVAL: 94297	NetSPA: 20526 TVA: 549448 MulVAL: 92005
	Number strong components	NetSPA: 110 TVA: 7960 MulVAL: 11618	**NetSPA: 1488** **TVA: 8580** **MulVAL: 12063**	NetSPA: 160 TVA: 8441 MulVAL: 11769
	Centrality	NetSPA: 0.5 TVA: 0.4 MulVAL: 0.1	NetSPA: 0.8 TVA: 0.8 MulVAL: 0.1	NetSPA: 0.6 TVA: 0.6 MulVAL: 0.1



(a) Path computation time diversity 0. (b) Path computation time diversity 0.5. (c) Path computation time diversity 1.

Fig. 7. Attack path computation time from one source to one target, according to the NetSPA model, mesh topology, uniform distribution, and vulnerability diversity (a) 0, (b) 0.5, and (c) 1. The highest bars correspond to impractical computation.

study the attack path computation time in relation to the number of hosts and vulnerabilities, according to three different values of vulnerability diversity, with the mesh topology and uniform distribution of vulnerabilities. However, it is worth noting that other settings do not show significant differences in values and trends.

Fig. 7 presents a baseline analysis for the attack path computation. To this aim, it reports the NetSPA model as having the best performance, considering the paths from one attack source to one target, randomly selected. The horizontal axes of the plots represent the number of hosts and the number of vulnerabilities, while the vertical axis represents the computation time in seconds. The bar colors highlight the cases when the scalability is good (less than 1 h), reasonable (between 1 and 2 h), and impractical (more than 2 h). The impractical experiments are capped at 7200 s, justifying the apparent stability of their trend. We can notice that when diversity is 0 (Fig. 7(a)), most of the cases have good scalability when the number of vulnerabilities is less than 100 and the number of hosts is less than 200. When the diversity is 0.5 (Fig. 7(b)), the number of cases when the computation is impractical increases three times, and the results are good when the number of vulnerabilities per host is less than 50 and reasonable when it is less than 100. Also, they are good when the number of hosts is less than 50. The experiments have the worst performance with diversity 1 (Fig. 7(c)), with tractable computation only when the number of hosts and vulnerabilities is less than 50, and good with only 25 hosts or vulnerabilities. Let us note that these conditions are met just for small-sized networks that are robust in terms of the number of exploitable vulnerabilities (Holm et al., 2011). In other words, in most of nowadays real-world networks, the problem is impractical. Another interesting result from Fig. 7 is that very few experiments present reasonable scalability (when diversity is 0, none of them), while they are either good or impractical. This highlights the exponential speed of performance degradation given the explosion of attack paths with hosts, vulnerabilities, and diversity.

Result 4: *The scalability of attack path computation degrades exponentially in the number of hosts, vulnerabilities, and diversity. This leads to a triple-exponential when varying them together. When the number of hosts and vulnerabilities exceeds 150, the problem becomes impractical with existing solutions.*

RQ5: ***What is the interplay between vulnerability distribution and vulnerability diversity?***

Given the impact of vulnerabilities, we explore how their distribution and diversity interplay. Their combination identifies the different security perimeters of a network, i.e., the boundaries to protect subnetworks (e.g., DMZ) from cyber attacks. To perform this analysis, we consider a LAN-based network with 100 hosts and 100 vulnerabilities per host, where the DMZ is central in the network. In this way, the centrality-based Pareto distribution indicates that the DMZ is the most vulnerable part of the network, and the vulnerability diversity indicates the heterogeneity of its servers: diversity 0 implies the DMZ contains all replicas, when it is 1, there are no replicas at all. The peripheral-based

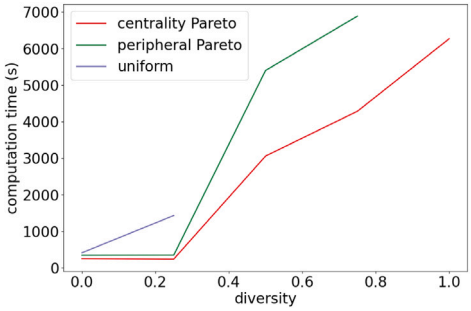
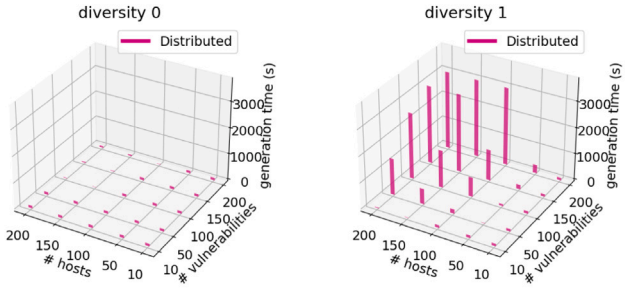


Fig. 8. Path computation for vulnerability distributions and diversity. Line discontinuity indicates impractical cases.



(a) Computation diversity 0. (b) Computation diversity 1.

Fig. 9. Path computation time per number of hosts and vulnerabilities and vulnerability diversity (a) 0 and (b) 1.

Pareto distribution indicates that the most vulnerable components in the network are the hosts (e.g., end-users), and the diversity indicates the heterogeneity of their services. For example, when diversity is 0, all the users run the same services (e.g., call center operators), when it is 1, the network hosts run different services (e.g., heterogeneous ICT). The uniform distribution indicates a network without formalized perimeter security (i.e., uniformly distributed vulnerabilities).

While there are no particular differences in the space analysis in terms of the number of nodes and edges, as also shown in Table 5, we focus on the analysis of computation time. Fig. 8 reports the path computation time, from one attack source to one attack target randomly selected, in the scenarios of different vulnerability distributions and diversity. As before, we consider the computation impractical for more than two hours of computation.

From the plot, it is visible that the attack paths can be computed when the diversity is 0 for any of the three distributions, while the computation becomes impractical when diversity is 0.5 for uniform distribution and when it is 0.75 for peripheral-based Pareto distribution (the line discontinuity in Fig. 8 represent the impractical cases). In contrast, the centrality-based Pareto distribution is tractable for any value of vulnerability diversity, showing a slower increase in the computation time than other distributions. This demonstrates that when the peripheral hosts are more protected (i.e., vulnerabilities are mostly in the center of the network), the path computation scales better in any diversity configuration from 0 to 1, with particular advantages for high diversity values. This is because most of the paths pass through the same central nodes, reducing the number of combinations of single steps and therefore the computed paths. As a consequence, the analysis of this interplay provides an interesting insight for network security: defining network perimeters not only gives the advantage in planning security strategies as stated in different security standards (ISO Central Secretary, 2022; ITILv4, 2019) (e.g., specific for the DMZ), but it also has the advantage that the resulting AG is more scalable in terms of computable attack paths.

Result 5: *Vulnerability distribution and diversity comparably affect the AG scalability, showing dependency with the perimeter security. Protecting peripheral hosts with critical services being central in the network allows for computing attack paths across any diversity configuration. Alternative network perimeters degrade performance, making computation impractical for diversity values exceeding 0.5.*

RQ6: ***How do the scalability results of the AG pillars apply to modern approaches?***

As a final investigation, we show that the results of the previous RQs apply to modern approaches as well. Considering the systematization of Table 2, we notice that *three* different solutions are currently considered.

(i) *Heuristic* approaches assign priorities to nodes and generate the AG accordingly (Sun et al., 2021; Feng et al., 2017; Wang et al., 2016). Priorities are predefined or calculated through metrics, such as vulnerability impacts or nodes' centrality. This provides only a different order of generation, while the complete algorithm falls into the same scenarios analyzed in the previous RQs.

Table 6
Distribution analysis in distributed approaches.

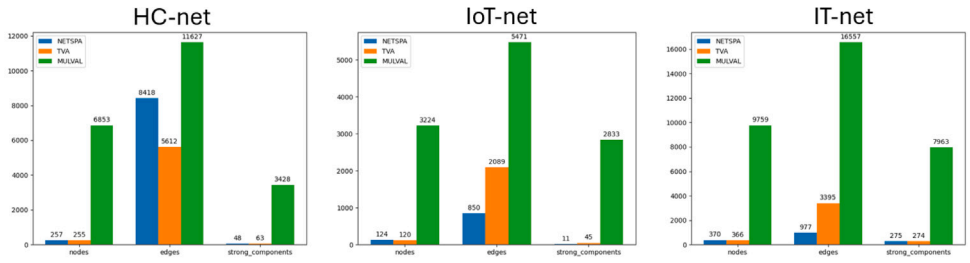
	Uniform distribution	Centrality Pareto	Peripheral Pareto
Model computation time	722.91 s	937.81 s	829.72 s
Path computation time	>2 h	~1.5 h	>2 h

(ii) Similarly, *alert-based* and *AI-based* generation considers only the assets (i.e., network hosts) that received an alert from monitoring systems (Hong and Kim, 2016; Ibrahim et al., 2019; Nadeem et al., 2022). In this case, only a sub-portion of the complete AG is generated, thus introducing the problem of incompleteness. As we focus on scalability, AG completeness is out of the scope of this paper, and we consider its analysis in future works.

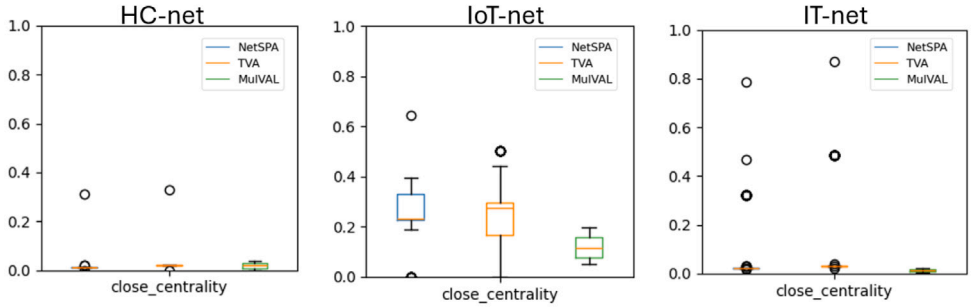
(iii) A final category includes algorithmic approaches based on *distributed computing* (Kaynar and Sivrikaya, 2016; Li et al., 2017; Mjihil et al., 2017; Sabur et al., 2022). They design algorithms to partition the AG, distribute the generation of the different partitions, and finally merge the results. To investigate the applicability of the previous findings to these approaches, we consider here the algorithm proposed by Sabur et al. (2022), as it is the most recent and a good representative of the other distributed solutions given the similarity of agents management with other approaches (Kaynar and Sivrikaya, 2016; Chen et al., 2020; Mjihil et al., 2017). We focus this analysis on the computation time, as the space remains the same in case the data is stored from a single agent. Let us note that in the case of distributed storage with replication, the space complexity increases even more. Fig. 9 shows the AG generation time varying the number of hosts (x-axis) and vulnerabilities (y-axis) with LAN-based topology and uniform distribution of the vulnerabilities.

It presents two different scenarios: when vulnerability diversity is 0 (Fig. 9(a)) and when it is 1 (Fig. 9(b)). It shows that both the trend and the time values are comparable to the analysis performed with the three pillar models (i.e., Fig. 5 and Fig. 7), thus indicating that the number of hosts, vulnerabilities, and vulnerability diversity all impact the AG scalability, even in modern (distributed) approaches. Looking at the experiments with diversity 0 (Fig. 9(a)), it is worth noting that settings with fewer hosts and vulnerabilities have higher times than the ones with more hosts and vulnerabilities (over $\tilde{1}50$). This is the effect of the overhead due to graph merging to rebuild the complete graph from different agents. In fact, as different works state (Kaynar and Sivrikaya, 2016; Mjihil et al., 2017), distributed approaches are disadvantageous for small network settings, as the communication overhead degrades the performance with respect to serial algorithms.

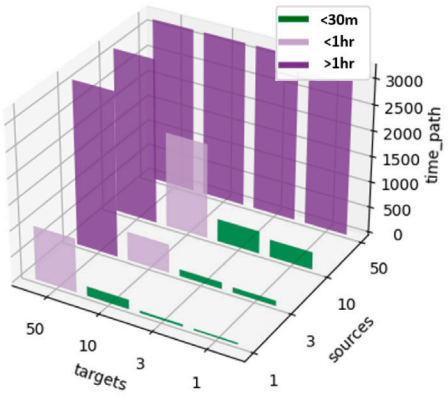
On the other hand, it is very interesting to observe that vulnerability diversity is invariant to this consideration. Fig. 9(b) shows that if



(a) Space complexity analysis for the real networks.



(b) Nodes' centrality in the real networks.



(c) Computation time by attack sources and targets in the best-case network.

Fig. 10. (a) Space complexity, (b) nodes' centrality distribution, and (c) time complexity analysis of the real networks.

we increase the diversity, it is not true anymore the advantage of distributed approaches for large networks. This highlights even more the need to evaluate AG algorithms over multiple parameters beyond network scenarios' size. Table 6 reports additional details concerning different vulnerability distributions for distributed approaches. It shows that vulnerability distribution affects the AG scalability with the same behavior analyzed for the three pillars (see Table 5), both for AG model generation and attack path computation. The better performance of centrality-based Pareto distribution with respect to uniform and peripheral-based distributions underlines that vulnerability distribution affects the scalability in distributed approaches as well.

Result 6: *The scalability findings apply to modern approaches. Heuristics and alert-based solutions introduce the problem of AG accuracy. Distributed approaches generate the complete AG, and the results of the previous RQs are still valid even considering more recent approaches. The number of hosts and vulnerabilities has a similar impact on the scalability (RQ1), as well as diversity matters (RQ2). Vulnerability distribution affects the scalability, with the same behavior analyzed for the three pillars (RQ3) and attack path computation (RQ4). The interplay between vulnerability diversity and distribution is a direct consequence of the previous analyses (RQ5).*

6. Empirical evaluations on real networks

In this section, we present an empirical evaluation of the scalability using data from the real networks described in Section 4.1. For the sake of completeness, the complementary risk assessment is in Appendix. This scalability analysis aims to quantify the space and time complexity of NetSPA, TVA, and MulVAL when applied in real scenarios. For the former, we analyze the number of nodes, edges, and strongly connected components; for the latter, we measure the time, in seconds, necessary to compute the attack paths, varying the number of attack sources and targets. Fig. 10 reports these analyses. The space complexity analysis (Fig. 10(a)) highlights the increased complexity of the logical AG (MulVAL) over the topological ones. The number of nodes in MulVAL is, on average, 26 times the number in NetSPA and TVA, while the number of edges is almost double. The reason for this higher complexity resides in the logical AGs expressing the network topology and vulnerability inventories in a more fine-grained manner, defining facts for the network status of single open ports and services. Concerning the number of strongly connected components, Fig. 10(a) shows that NetSPA has the fewest, while MulVAL has the highest. This indicates that in NetSPA, we can expect a slower degradation of attack path computation than the other models. This outcome aligns with our

expectation, as the NetSPA model includes *prerequisite* nodes, which are usually fewer than hosts and vulnerabilities nodes, resulting in an increased graph centrality on them. Let us recall that we are interested in highlighting the closeness centrality as it informs about the number of combinations of different attack steps, given that most of the paths pass through the central nodes (see Section 4.2). Consequently, most of the attack paths traverse these highly central nodes, thereby decreasing the potential combinations of attack steps. Fig. 10(b) confirms this outcome, showing that NetSPA has higher and less variable nodes' closeness centrality.

Regarding the time complexity, Fig. 10(c) shows the time (in seconds) needed to compute the attack paths varying the number of attack sources and targets, chosen randomly among the AG nodes, and according to the NetSPA model on the HC-net which shows the best-case performance (i.e., the lower bound). It shows that we can compute attack paths with one source and one target in a few seconds; if either the sources are at most 10 and the targets at most 3 (or vice-versa), it takes about 8 min; if we increase further the number of sources and targets, the computation becomes impractical. This means that the attack path computation can be leveraged for risk analysis only with a few specific attack sources and targets, while it is impractical even for simple analyses (e.g., computing attack paths for a target sub-network with more than 10 hosts).

Let us note that the attack path computation with TVA and MulVAL is impractical, requiring more than three hours of computation to compute all the paths. The reason why NetSPA outperforms the other model in attack paths computation can once again be attributed to the closeness centrality of its nodes (Fig. 10(b)). NetSPA concentrates its centrality on a few prerequisite nodes, and most paths traverse these nodes. The closeness centrality of the remaining nodes is represented as outliers (the small circles in the boxplot of Fig. 10(b)), signifying their comparatively diminished impact on centrality. Conversely, TVA and MulVAL do not have outliers in the distribution of closeness centrality, meaning that all nodes participate in attack paths without any nodes being excessively central or peripheral. The drawback of this outcome is the high number of attack step combinations during the path generation.

In summary, this empirical evaluation confirms that the scalability parameters in the proposed framework align with real scenarios. This emphasizes the need for considering additional parameters beyond network size when analyzing AG-based algorithms to avoid impractical computations in real networks: proper management of AG generation is crucial for practical use in dynamic networks, especially in real-time environments.

7. Discussion

This paper proposed a vulnerable network generator to test AG algorithms resembling real networks. This enabled the design of an analytical framework for an extensive experimental evaluation of AG scalability, highlighting the root causes of AG scalability degradation. The experimental evaluation highlighted that network size is not the only parameter degrading the performance of AG algorithms, nor the most impacting in absolute terms. We experimentally proved that the number of vulnerabilities affects the AG scalability similarly to the number of hosts (RQ1), the vulnerability diversity and network topology may result in a sixfold degradation of AG complexity (RQ2), and vulnerability distribution and placement play a central role in both space and time complexity (RQ3). Additionally, the attack path computation is impractical for networks with reasonable configurations (RQ4), and the higher exposure of peripheral hosts negatively impacts the AG scalability (RQ5). While the in-depth evaluation has been performed for the three pillars of AG models, the findings still apply to modern approaches, in particular distributed ones (RQ6). When not considering these parameters comprehensively, experimental validation may result in incomplete or inaccurate scalability evaluation. For example, the best configuration (few hosts, few vulnerabilities, vulnerability diversity 0, tree topology, and centrality-based distribution) and the worst one (many hosts, many vulnerabilities, vulnerability diversity 1, mesh topology, and uniform distribution) may lead to completely different results: good scalability in the former case, impractical in the latter. In this section, we discuss how our findings can benefit future AG algorithms and their applicability to real networks.

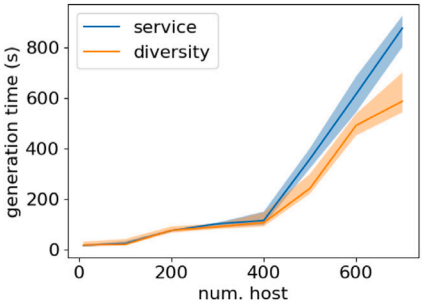


Fig. 11. AG generation for different distribution strategies.

7.1. Informativeness and benefits of the results

One of the main goals of this paper is to provide findings to inform more scalable AG algorithms. To show an example of such informativeness, let us consider the distributed algorithms, which are the current cutting-edge approaches for AG generation (Sabur et al., 2022; Palma and Bonomi, 2023). As discussed in RQ6, the existing solutions distribute the generation according to the services in the hosts, and we showed that it may not always be the most efficient choice, as other parameters affect the AG scalability. To this aim, we perform a preliminary comparison of the AG generation distributed according to *service* and vulnerability *diversity*, given its high impact on scalability. Fig. 11 reports the preliminary investigation of the generation time. It shows that distributing on vulnerability diversity is more advantageous than on services, saving about 25% of the computation time. This is mainly due to the fastest computation of each agent, thanks to similar vulnerabilities (see RQ2). Thus, the first usage of the proposed findings is to inform more scalable distributed AG generation approaches

Another possibility to leverage the results of this paper is to design a context-aware algorithm that, given a vulnerable network as input, detects partitions according to topology, vulnerability distribution, and diversity, so as to optimize the computation in each partition. For example, DMZ is supposed to be heterogeneous in terms of vulnerability diversity, while a data center may have a low degree of vulnerability diversity because of multiple replicas for backup. Similarly, the DMZ is supposed to be structured in a star-like topology, while the storage into a mesh-like topology. Thus, the algorithm may determine the topology-vulnerability trade-off and balance the AG generation in the sub-networks accordingly. Possible ways to investigate such a trade-off include training a Machine Learning (ML) model with the data from the experimental evaluation made available in this paper.

Finally, after performing a complementary investigation on the AG algorithm's completeness and accuracy, which we are planning for future works, the findings of this paper can inform a comprehensive decision support system where different AG algorithms are combined. For example, in subnetworks with a star topology, a topological AG is scalable enough, while in subnetworks with low diversity, a distributed solution is more appropriate.

7.2. Consideration of real networks

An interesting point of discussion from our research is that the literature lacks an investigation of the features of *real* networks and

vulnerability inventories for AG scalability analysis. The main reason for this lack is that real networks include sensitive information difficult to retrieve because of privacy. In this work, we moved a step towards this problem by proposing both a generator of vulnerable networks and anonymized features of three real networks. It represents a baseline for this type of dataset and opens the opportunity to research innovative solutions to this problem. For example, alternative approaches may leverage generative neural networks trained on real network features to generate vulnerable networks. Additionally, the dataset generator provides useful data for other cybersecurity tasks, such as vulnerability management, network topology-related problems, as well as detection problems. Finally, Section 6 shows that the findings of the experimental evaluation apply to the real networks as well, as we expected given the similarity analysis of the synthetic vulnerable networks and the real ones.

8. Conclusion

This paper proposed a framework for an extensive root cause analysis of AG scalability, showing the impact of (i) number of hosts, (ii) vulnerabilities, (iii) vulnerability diversity, (iv) network topology, and (v) vulnerability distribution. We investigated synthetic and real networks, highlighting their similarities and the effectiveness of the proposed framework and dataset generator. In future works, we will extend the investigation by defining common testing configurations in real networks, as the authors did not find other publicly available datasets for similar problems at the time of this writing. In addition, we are currently working on deriving a theoretical scalability model from our experimental evaluation that can be used to predict the performance of the AG generation process for a given input network. Finally, we will design a scalable AG generation algorithm by applying the findings of this paper. For example, this benchmark could help the design of dynamic approaches, where network and vulnerability distribution may change dynamically.

CRediT authorship contribution statement

Alessandro Palma: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Silvia Bonomi:** Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to thank Andrea Agiollo and Lorenzo Di Filippo for their valuable support, constructive feedback, and assistance in testing the code. Their contributions were instrumental in enhancing the reliability and performance of our implementation.

This work was partially supported by project SERICS (PE00000014) under the MUR National Recovery and Resilience Plan funded by the European Union - NextGenerationEU and by the *Avvio alla Ricerca* Sapienza grant AR2241902B426269.

Appendix. Risk assessment of real networks

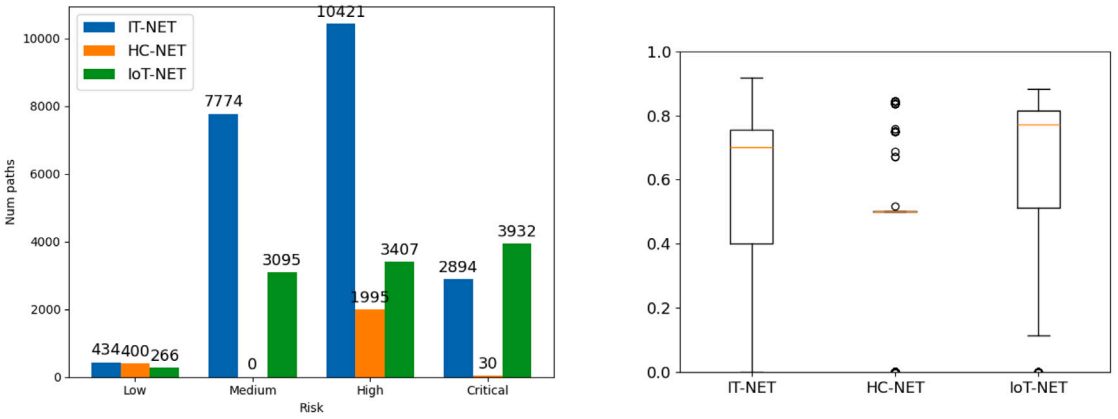
In this Appendix, we complement the analysis of the real networks through their risk assessment. Fig. A.12 reports the results of risk assessment for the IT department (IT-net), healthcare infrastructure (HC-net), and IoT network (IoT-net), whose details are in Section 6.

Each of these networks serves a different function and thus operates under different constraints and levels of criticality. In fact, Fig. A.12(a) quantifies the number of attack paths across various risk levels (i.e., low, medium, high, and critical risk), while Fig. A.12(b) reports the distribution of the risk values. The IT-net stands out immediately due to the sheer number of attack paths identified, which are more than 21,000 in total. Most notably, the largest concentration is in the high-risk category, with more than 10,000 attack paths. This is followed by nearly 8000 paths in the medium-risk range, and a non-negligible number (2894) even fall into the critical-risk range. This pattern reflects a broad and highly varied attack surface. Universities typically have open environments to facilitate research, collaboration, and access for students and staff. While this openness is essential to the institution's mission, it often results in a heterogeneous and less tightly controlled infrastructure, making consistent security enforcement challenging. The corresponding box plot in Fig. A.12(b) further supports this, showing a wide spread in risk value ranging from low to high, which aligns with the idea that different parts of the network have vastly different levels of exposure and risk.

The IoT network presents a total number of attack paths lower than that of IT-net, with around 10,700 paths identified. Looking at the distribution of these paths, we can notice that a large portion falls into the high and critical categories, with nearly 4000 attack paths classified as critical alone. This suggests a pervasive and systemic level of exposure within the IoT environment. The box plot backs this up, showing a high median risk value (close to 0.8) with a relatively narrow interquartile range. This means that most paths have consistently high risk levels, unlike IT-NET's wider variability. The implication is that the IoT network's vulnerabilities are not limited to isolated segments; rather, they are spread throughout the system, likely due to common IoT weaknesses such as default credentials, outdated firmware, and lack of proper segmentation.

In contrast, the healthcare infrastructure has a relatively small number of attack paths, under 2500 in total. However, it is important not to misinterpret this as a sign of robust security. While the number of attack vectors is limited, there are still nearly 2000 paths classified as high-risk and a small but significant number (30) that are critical. This is particularly concerning given that healthcare networks are classified as critical infrastructure, meaning that even a single successful exploit can have severe consequences, including risks to patient safety and service disruption. The box plot reveals that risk scores in the healthcare network are tightly clustered around the median, suggesting a uniform risk profile. On the surface, this may indicate more centralized or consistent risk management practices. However, the presence of high and critical risk paths in a network that must meet strict reliability and safety standards underscores a need for targeted, risk-based mitigation, especially for those high-severity vulnerabilities that still exist despite the overall smaller attack surface.

When comparing the three networks, we can observe that the IT-net is likely reflective of an environment that lacks strict enforcement but is not mission-critical. Its risk variability offers opportunities for risk prioritization and segmentation. The healthcare network, as a critical infrastructure, should ideally have no critical or high-risk attack paths, yet the data reveals that these do exist. Finally, the IoT network emerges as the most uniformly exposed and systematically vulnerable network. Its high number of critical paths, combined with a consistently high-risk profile, points to deep-rooted architectural and operational weaknesses that likely stem from poor IoT security standards and widespread device heterogeneity. This comparative analysis highlights that risk management must be tailored not only to the threat landscape but also to the functional context and criticality of the network itself.



(a) Number of paths for low, medium, high, and critical risks for the three real networks.

(b) Boxplots of risk values for the three real networks.

Fig. A.12. Cyber risk assessment for the three real networks.

Data availability

Data and code are openly available at the following link: https://github.com/ds-square/vuln-net-ag. Additional data concerning real networks cannot be disclosed as they include sensitive information. Authors may provide further information about the dataset upon request.

References

Amankwah, R., Chen, J., Kudjo, P.K., Towey, D., 2020. An empirical comparison of commercial and open-source web vulnerability scanners. Softw.: Pr. Exp. 50 (9), 1842–1857. http://dx.doi.org/10.1002/spe.2870, arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.2870, URL https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.2870.

Angelini, M., Bonomi, S., Lenti, S., Santucci, G., Taggi, S., 2019. MAD: A visual analytics solution for Multi-step cyber Attacks Detection. J. Comput. Lang. 52, 10–24.

Barik, M.S., Sengupta, A., Mazumdar, C., 2016. Attack graph generation and analysis techniques. Def. Sci. J. 66 (6), 559. http://dx.doi.org/10.14429/dsj.66.10795, URL http://publications.drdo.gov.in/ojs/index.php/dsj/article/view/10795.

Bonomi, S., Cuoci, M., Lenti, S., Palma, A., 2025. Improving attack graph-based self-protecting systems: A computational pipeline for accuracy-scalability trade-off. In: Collart-Dutilleul, S., Ouchani, S., Cuppens, N., Cuppens, F. (Eds.), Risks and Security of Internet and Systems. Springer Nature Switzerland, Cham, pp. 525–542. http://dx.doi.org/10.1007/978-3-031-89350-6_33.

Chen, Y., Liu, Z., Liu, Y., Dong, C., 2020. Distributed attack modeling approach based on process mining and graph segmentation. Entropy 22 (9), 1026. http://dx.doi.org/10.3390/e22091026, Number: 9 Publisher: Multidisciplinary Digital Publishing Institute, URL https://www.mdpi.com/1099-4300/22/9/1026.

Coleman, T.F., Moré, J.J., 1983. Estimation of sparse Jacobian matrices and graph coloring blems. SIAM J. Numer. Anal. 20 (1), 187–209.

Ding, W., Chen, B., Li, B., Eun, K.J., Zhao, D., 2021. Multimodal safety-critical scenarios generation for decision-making algorithms evaluation. IEEE Robot. Autom. Lett. 6 (2), 1551–1558.

Erdős, P., Rényi, A., 1959. On random graphs I. Publ. Math. Debrecen 6 (290–297), 18.

Feng, Y., Wang, L., Zhang, J., Cai, Z., Gan, Y., 2017. Generation method of network attack graph based on greedy heuristic algorithm. Int. J. Hybrid Inf. Technol. 10 (6), 23–32. http://dx.doi.org/10.14257/ijhit.2017.10.6.03, URL http://gvpress.com/journals/IJHIT/vol10_no6/3.pdf.

Gonzalez-Granadillo, G., Dubus, S., Motzek, A., Garcia-Alfaro, J., Alvarez, E., Merialdo, M., Papillon, S., Debar, H., 2018. Dynamic risk management response system to handle cyber threats. Future Gener. Comput. Syst. 83, 535–552.

Griner, C., Zerwas, J., Blenk, A., Ghobadi, M., Schmid, S., Avin, C., 2021. Cerberus: The power of choices in datacenter topology design - a throughput perspective. Proc. ACM Meas. Anal. Comput. Syst. (SIGMETRICS) 5 (3), http://dx.doi.org/10.1145/3491050.

Hashmat, F., Abbas, S.G., Hina, S., Shah, G.A., Bakhshi, T., Abbas, W., 2022. An automated context-aware IoT vulnerability assessment rule-set generator. Comput. Commun. 186, 133–152. http://dx.doi.org/10.1016/j.comcom.2022.01.022.

Heberlein, T., Bishop, M., Ceesay, E., Danforth, M., Senthilkumar, C., Stallard, T., 2012. A taxonomy for comparing attack-graph approaches. [Online] https://www.academia.edu/6146967/A_Taxonomy_for_Comparing_Attack_Graph_Approaches.

Holm, H., Sommestad, T., Almroth, J., Persson, M., 2011. A quantitative evaluation of vulnerability scanning. Inf. Manage. Comput. Secur. 19 (4), 231–247.

Hong, J.B., Kim, D.S., 2016. Towards scalable security analysis using multi-layered security models. J. Netw. Comput. Appl. 75, 156–168. http://dx.doi.org/10.1016/j.jnca.2016.08.024, URL https://www.sciencedirect.com/science/article/pii/S1084804516301928.

Ibrahim, A., Bozhinoski, S., Pretschner, A., 2019. Attack graph generation for microservice architecture. In: Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing. pp. 1235–1242.

Ingols, K., Lippmann, R., Piwowarski, K., 2006. Practical attack graph generation for network defense. In: 2006 22nd Annual Computer Security Applications Conference. ACSAC'06, pp. 121–130. http://dx.doi.org/10.1109/ACSAC.2006.39 (ISSN: 1063-9527).

ISO Central Secretary, 2022. ISO/IEC 27001:2022-Information Security, Cybersecurity and Privacy Protection - Guidance on Managing Information Security Risks. Standard ISO/IEC 27005:2022, International Organization for Standardization, Geneva, CH.

ITILv4, 2019. Information Technology Infrastructure Library, Vol. 2019. Standard, Axelos, UK.

Jain, R., 1991. The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling, vol. 1, Wiley New York.

Jain, A., Dwivedi, R.K., Alshazly, H., Kumar, A., Bourouis, S., Kaur, M., 2022. Design and simulation of ring network-on-chip for different configured nodes. Comput. Mater. Contin. 71 (2), 4085–4100.

Jajodia, S., Noel, S., O'berry, B., 2009. Topological analysis of network attack vulnerability. In: Managing Cyber Threats: Issues, Approaches, and Challenges. Springer, pp. 247–266.

Kaynar, K., 2016. A taxonomy for attack graph generation and usage in network security. J. Inf. Secur. Appl. 29, 27–56. http://dx.doi.org/10.1016/j.jisa.2016.02.001, URL https://www.sciencedirect.com/science/article/pii/S2214212616300011.

Kaynar, K., Sivrikaya, F., 2016. Distributed attack graph generation. IEEE Trans. Dependable Secur. Comput. 13 (5), 519–532. http://dx.doi.org/10.1109/TDSC.2015.2423682, Conference Name: IEEE Transactions on Dependable and Secure Computing.

Knight, S., Nguyen, H.X., Falkner, N., Bowden, R., Roughan, M., 2011. The internet topology zoo. IEEE J. Sel. Areas Commun. 29 (9), 1765–1775.

Lee, C.Y., 1961. An algorithm for path connections and its applications. IRE Trans. Electron. Comput. (3), 346–365.

Li, M., Hawrylak, P.J., Hale, J., 2020. Implementing an attack graph generator in CUDA. In: 2020 IEEE International Parallel and Distributed Processing Symposium Workshops. IPDPSW, pp. 730–738. http://dx.doi.org/10.1109/IPDPSW50202.2020.00128.

Li, T., Jiang, Y., Lin, C., Obaidat, M., Shen, Y., Ma, J., 2022a. DeepAG: Attack graph construction and threats prediction with bi-directional deep learning. IEEE Trans. Dependable Secur. Comput. http://dx.doi.org/10.1109/TDSC.2022.3143551, 1–1, Conference Name: IEEE Transactions on Dependable and Secure Computing.

Li, H., Wang, Y., Cao, Y., 2017. Searching forward complete attack graph generation algorithm based on hypergraph partitioning. Procedia Comput. Sci. 107, 27–38.

Li, Z., Zeng, J., Chen, Y., Liang, Z., 2022b. AttacKG: Constructing technique knowledge graph from cyber threat intelligence reports. In: European Symposium on Research in Computer Security. Springer, pp. 589–609.

Lippmann, R.P., Ingols, K.W., Scott, C., Piwowarski, K., Kratkiewicz, K., Artz, M., Cunningham, R., 2005. Evaluating and Strengthening Enterprise Network Security Using Attack Graphs. Tech. Rep., Citeseer.

Mannan, R.W., Meslow, E.C., 1984. Bird populations and vegetation characteristics in managed and old-growth forests, northeastern Oregon. J. Wildl. Manag. 1219–1238.

Mjihil, O., Huang, D., Haqiq, A., 2017. Improving attack graph scalability for the cloud through SDN-based decomposition and parallel processing. In: Sabir, E., García Armada, A., Ghogho, M., Debbah, M. (Eds.), Ubiquitous Networking. In: Lecture Notes in Computer Science, Springer International Publishing, Cham, pp. 193–205.

Muñoz-González, L., Sgandurra, D., Barrère, M., Lupu, E.C., 2019. Exact Inference Techniques for the Analysis of Bayesian Attack Graphs. IEEE Trans. Dependable Secur. Comput. 16 (2), 231–244. http://dx.doi.org/10.1109/TDSC.2016.2627033, Conference Name: IEEE Transactions on Dependable and Secure Computing.

Nadeem, A., Verwer, S., Moskal, S., Yang, S.J., 2022. Alert-driven attack graph generation using S-PDFA. IEEE Trans. Dependable Secur. Comput. 19 (2), 731–746. http://dx.doi.org/10.1109/TDSC.2021.3117348, Conference Name: IEEE Transactions on Dependable and Secure Computing.

Navarro, J., Deruyver, A., Parrend, P., 2018. A systematic survey on multi-step attack detection. Comput. Secur. 76, 214–249.

Noel, S., Purdy, S., O'Rourke, A., Overly, E., Chen, B., DiFonzo, C., Chen, J., Sakellis, G., Hegde, M., Sapra, M., et al., 2023. Graph analytics and visualization for cyber situational understanding. J. Déf. Model. Simul. 20 (1), 81–95.

Open Web Application Security Project, 2021. OWASP Top Ten, Vol. 2021. Standard, OWASP Foundation.

Ou, X., Govindavajhala, S., Appel, A.W., et al., 2005. MulVAL: A logic-based network security analyzer. In: USENIX Security Symposium, Vol. 8. Baltimore, MD, pp. 113–128.

Palma, A., Angelini, M., 2024. It is time to steer: A scalable framework for analysis-driven attack graph generation. In: Garcia-Alfaro, J., Kozik, R., Choraś, M., Katsikas, S. (Eds.), Computer Security. ESORICS 2024, Springer Nature Switzerland, Cham, pp. 229–250.

Palma, A., Bonomi, S., 2023. A workflow for distributed and resilient attack graph generation. In: 2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks-Supplemental Volume. DSN-S, IEEE, pp. 185–187.

Palma, A., Papadakis, N., Bouloukakis, G., Garcia-Alfaro, J., Sospetti, M., Magoutis, K., 2025. SHIELD: assessing security-by-design in federated data spaces using attack graphs. In: The 40th ACM/SIGAPP Symposium on Applied Computing. SAC.

Palmer, C.R., Steffan, J.G., 2000. Generating network topologies that obey power laws. In: Globecom'00-IEEE. Global Telecommunications Conference. Conference Record (Cat. No. 00CH37137), vol. 1, IEEE, pp. 434–438.

Presekal, A., Ştefanov, A., Rajkumar, V.S., Palensky, P., 2023. Attack graph model for cyber-physical power systems using hybrid deep learning. IEEE Trans. Smart Grid 14 (5), 4007–4020. http://dx.doi.org/10.1109/TSG.2023.3237011.

Rodler, M., Paaßen, D., Li, W., Bernhard, L., Holz, T., Karame, G., Davi, L., 2023. EF-CF: High performance smart contract fuzzing for exploit generation. In: 2023 IEEE 8th European Symposium on Security and Privacy. EuroS&P, pp. 449–471. http://dx.doi.org/10.1109/EuroSP57164.2023.00034.

Sabidussi, G., 1966. The centrality index of a graph. Psychometrika 31 (4), 581–603.

Sabur, A., Chowdhary, A., Huang, D., Alshamrani, A., 2022. Toward scalable graph-based security analysis for cloud networks. Comput. Netw. 206, 108795. http://dx.doi.org/10.1016/j.comnet.2022.108795, URL https://www.sciencedirect.com/science/article/pii/S1389128622000251.

Sadlek, L., Čeleda, P., Tovarňák, D., 2022. Identification of attack paths using kill chain and attack graphs. In: NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium. IEEE, pp. 1–6.

Salayma, M., 2024. Threat modelling in Internet of Things (IoT) environments using dynamic attack graphs. Front. the Internet Things 3, 1306465.

Schreuders, Z.C., Shaw, T., Shan-A-Khuda, M., Ravichandran, G., Keighley, J., Ordean, M., 2017. Security scenario generator (SecGen): A framework for generating randomly vulnerable rich-scenario VMs for learning computer security and hosting CTF events. In: 2017 USENIX Workshop on Advances in Security Education. ASE 17, USENIX Association, Vancouver, BC, URL https://www.usenix.org/conference/ase17/workshop-program/presentation/schreuders.

Sheng, X.-R., Zhao, L., Zhou, G., Ding, X., Dai, B., Luo, Q., Yang, S., Lv, J., Zhang, C., Deng, H., et al., 2021. One model to serve all: Star topology adaptive recommender for multi-domain ctr prediction. In: Proceedings of the 30th ACM International Conference on Information & Knowledge Management. pp. 4104–4113.

Sichitiu, M.L., 2005. Wireless mesh networks: opportunities and challenges. In: Proceedings of World Wireless Congress, vol. 2, p. 21.

Stan, O., Bitton, R., Ezrets, M., Dadon, M., Inokuchi, M., Ohta, Y., Yagyu, T., Elovici, Y., Shabtai, A., 2022. Extending attack graphs to represent cyber-attacks in communication protocols and modern IT networks. IEEE Trans. Dependable Secur. Comput. 19 (3), 1936–1954. http://dx.doi.org/10.1109/TDSC.2020.3041999.

Stergiopoulos, G., Dedousis, P., Gritzalis, D., 2022. Automatic analysis of attack graphs for risk mitigation and prioritization on large-scale and complex networks in Industry 4.0. Int. J. Inf. Secur. 21 (1), 37–59.

Sun, W., Li, Q., Wang, P., Hou, J., 2021. Heuristic network security risk assessment based on attack graph. In: International Conference on Cloud Computing. Springer, pp. 181–194.

Tanimoto, T.T., 1958. Elementary Mathematical Theory of Classification and Prediction. International Business Machines Corp.

Tayouri, D., Baum, N., Shabtai, A., Puzis, R., 2023. A survey of mulval extensions and their attack scenarios coverage. IEEE Access.

Upadhyay, D., Sampalli, S., 2020. SCADA (Supervisory Control and Data Acquisition) systems: Vulnerability assessment and security recommendations. Comput. Secur. 89, 101666. http://dx.doi.org/10.1016/j.cose.2019.101666, URL https://www.sciencedirect.com/science/article/pii/S0167404819302068.

Venkatesan, S., Youzwak, J.A., Sugrim, S., Chiang, C.-Y.J., Poylisher, A., Witkowski, M., Walther, G., Wolberg, M., Chadha, R., Newcomb, E.A., Hoffman, B., Buchler, N., 2019. VulnerVAN: A vulnerable network generation tool. In: MILCOM 2019 - 2019 IEEE Military Communications Conference. MILCOM, pp. 1–6. http://dx.doi.org/10.1109/MILCOM47813.2019.9021013.

Wang, S., Tang, G., Kou, G., Chao, Y., 2016. An attack graph generation method based on heuristic searching strategy. In: 2016 2nd IEEE International Conference on Computer and Communications. ICCC, pp. 1180–1185. http://dx.doi.org/10.1109/CompComm.2016.7924891.

Yang, J., Yang, M., Han, W., Zhang, J., 2023. A survey on attack graph. In: International Conference on Network Simulation and Evaluation. Springer, pp. 36–52.

Yichao, Z., Tianyang, Z., Xiaoyue, G., Qingxian, W., 2019. An improved attack path discovery algorithm through compact graph planning. IEEE Access 7, 59346–59356.

Yousefi, M., Mtetwa, N., Zhang, Y., Tianfield, H., 2018. A reinforcement learning approach for attack graph analysis. In: 2018 17th IEEE International Conference on Trust, Security and Privacy in Computing and Communications/12th IEEE International Conference on Big Data Science and Engineering. TrustCom/BigDataSE, IEEE, pp. 212–217.

Yuan, B., Pan, Z., Shi, F., Li, Z., 2020. An attack path generation methods based on graph database. In: 2020 IEEE 4th Information Technology, Networking, Electronic and Automation Control Conference, Vol. 1. ITNEC, pp. 1905–1910. http://dx.doi.org/10.1109/ITNEC48623.2020.9085039.

Zager, L.A., Verghese, G.C., 2008. Graph similarity scoring and matching. Appl. Math. Lett. 21 (1), 86–94.

Zenitani, K., 2022a. Attack graph analysis: an explanatory guide. Comput. Secur. 103081.

Zenitani, K., 2022b. A scalable algorithm for network reachability analysis with cyclic attack graphs. J. Comput. Secur. Preprint (Preprint), 1–27. http://dx.doi.org/10.3233/JCS-210103, Publisher: IOS Press, URL https://ip.ios.semcs.net/articles/journal-of-computer-security/jcs210103.

Zhang, M., Li, X., Liu, B., Lu, J., Zhang, Y., Chen, J., Duan, H., Hao, S., Zheng, X., 2023. Detecting and measuring security risks of hosting-based dangling domains. Proc. ACM Meas. Anal. Comput. Syst. (SIGMETRICS) 7 (1), http://dx.doi.org/10.1145/3579440.

Zhang, Z., Ma, H., Wu, D., Gao, D., Yi, X., Chen, Y., Wu, Y., Jiang, L., 2024. MtdScout: Complementing the identification of insecure methods in android apps via source-to-bytecode signature generation and tree-based layered search. In: 2024 IEEE 9th European Symposium on Security and Privacy. EuroS&P, pp. 724–740. http://dx.doi.org/10.1109/EuroSP60621.2024.00045.



Alessandro Palma is a Post Doctoral researcher in Engineering in Computer Science in the Department of Computer, Control, and Management Engineering "Antonio Ruberti" at Sapienza University of Rome, Italy. He received the B.Sc. and M.Sc. in Engineering in Computer Science from the University of Rome Tor Vergata in 2018 and Sapienza University of Rome in 2020, respectively. He worked in 2021 as a research fellow with CINI Cyber Security National Laboratory on OSINT topics. In 2024, he was a visiting scholar at Télécom SudParis – IP Paris. His main research interests focus on autonomic cybersecurity and resilience and support for quantifiable cyber risk assessment, researching attack modeling and attack graphs.



Silvia Bonomi is Associate Professor at Sapienza University of Rome. She got a joint Ph.D. in Computer Science and Computer Engineering. She is member of Cyber Intelligence and Information Security research group (CIS-Sapienza) of the Department of Computer, Control, and Management Engineering "Antonio Ruberti" since 2012 and of the Midlab research group since 2006. She is also member of the CINI Cyber Security National Laboratory. She currently leads the Distributed Systems research group at DIAG Sapienza. She has been and is currently involved in several National (PNRR PE7, PNRR CN2, PRIN ESTEEM and PRIN TENACE) and EU projects (EU-GUARDIAN, PANACEA, Semantic-Gov, ReSIST, GreenerBuildings, eDIANA, PANOPTESEC). Her main research interests are in the context of secure and dependable distributed systems. In particular, she targets topics like attack modeling, fault tolerance (also considering malicious faults) and management of autonomic process behaviors in unmanaged environment (e.g. peer-to-peer systems). Other research interests are: information dissemination systems, event-based systems, data dissemination in sensors networks.